TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



NGUYỄN NGHĨA HIẾU – 52400013
TRẦN QUỐC BẢO – 52400102

ENSEMBLE LEARNING

BÁO CÁO GIỮA KỲ
NHẬP MÔN HỌC MÁY

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



NGUYỄN NGHĨA HIẾU – 52400013
TRẦN QUỐC BẢO – 52400102

ENSEMBLE LEARNING

BÁO CÁO GIỮA KỲ
NHẬP MÔN HỌC MÁY

Người hướng dẫn
TS. Trần Lương Quốc Đại

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn TS. Trần Lương Quốc Đại, giảng viên trường Đại học Tôn Đức Thắng, người hướng dẫn tận tình, góp ý và chỉ bảo cho chúng em trong việc hoàn thành bài báo cáo giữa kỳ của môn Nhập môn học máy. Nhờ sự chỉ bảo tận tình cũng như nhiệt huyết của giảng viên đã giúp chúng em hiểu rõ hơn về kiến thức chuyên môn của môn học và nhờ đó truyền cảm hứng để chúng em có thể học tập tốt hơn những môn liên quan sau này.

Chúng em cũng xin gửi lời cảm ơn đến toàn thể giảng viên của khoa Công nghệ thông tin nói riêng và toàn thể Giảng viên của trường Đại học Tôn Đức Thắng nói chung. Những điều kiện thuận lợi mà thầy cô tạo ra đã giúp chúng em củng cố kiến thức nền tảng và hoàn thành tốt việc học.

Dù đã nỗ lực hết sức trong việc hoàn thành bài báo cáo giữa kỳ này nhưng song song đó cũng có thể có những thiếu sót trong quá trình thực hiện và cũng như thiếu sót về mặt kiến thức. Chúng em rất mong nhận được sự góp ý cũng như đánh giá từ quý thầy cô để chúng em có thể hoàn thành tốt hơn không chỉ trong bài báo cáo này mà còn trong những bài báo cáo sau nữa. Chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, ngày 28 tháng 10 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)



Nguyễn Nghĩa Hiếu Trần Quốc Bảo

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH
TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Chúng em xin cam đoan đây là công trình nghiên cứu của nhóm chúng em và được sự hướng dẫn khoa học của TS. Trần Lương Quốc Đại. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Báo cáo còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào nhóm chúng em xin hoàn toàn chịu trách nhiệm về nội dung Báo cáo của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do chúng em gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 28 tháng 10 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)



Nguyễn Nghĩa Hiếu Trần Quốc Bảo

TÓM TẮT

Báo cáo trình bày Ensemble Learning, một hướng tiếp cận kết hợp nhiều mô hình nhằm tạo ra mô hình dự đoán tổng hợp. Nội dung tập trung vào Bagging cây quyết định, gồm lấy mẫu bootstrap có hoàn lại, huấn luyện các cây trên những mẫu khác nhau và kết hợp dự đoán bằng bỏ phiếu đa số.

Để làm rõ cơ chế hoạt động, báo cáo sử dụng một ví dụ gồm năm quan sát và ba cây quyết định. Ví dụ được triển khai xuyên suốt từ tạo bootstrap, tính độ hỗn tạp Gini, lựa chọn ngưỡng chia, dự đoán của từng cây đến kết quả bỏ phiếu. Phương pháp đánh giá out-of-bag cũng được minh họa để phân biệt những quan sát đã và chưa được mỗi cây sử dụng khi huấn luyện.

Phần thực nghiệm sử dụng bộ dữ liệu Breast Cancer Wisconsin Diagnostic gồm 569 quan sát và 30 đặc trưng số. Bagging tự triển khai được so sánh với cây quyết định đơn và Random Forest trên mười lần chia dữ liệu có phân tầng. Accuracy trung bình của Bagging đạt 94,76%, với độ lệch chuẩn mẫu 1,59 điểm phần trăm, cao hơn cây đơn trong phạm vi khảo sát.

Báo cáo bổ sung khảo sát số cây, phân tích mẫu dự đoán sai, đối chứng giữa cách xét đặc trưng và quy tắc tổng hợp, đồng thời minh họa AdaBoost, Voting và Stacking trên cùng mười lần chia dữ liệu. Các mô hình cơ sở cũng được đánh giá riêng nhằm kiểm tra lợi ích của việc kết hợp. Kết quả cho thấy tổ hợp không tự động vượt qua từng mô hình đơn; chất lượng phụ thuộc vào cấu hình mô hình cơ sở, dữ liệu và phương pháp tổng hợp. Các kết luận được diễn giải trong phạm vi dữ liệu và giao thức đã khảo sát.

Từ khóa: Ensemble Learning, Bagging, AdaBoost, Voting, Stacking, bootstrap, OOB, OOF.

MỤC LỤC

DANH MỤC HÌNH VẼ x

DANH MỤC BẢNG BIỂU xi

DANH MỤC THUẬT TOÁN xiii

DANH MỤC CHỮ VIẾT TẮT xiv

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI 1

1.1 Lý do thực hiện đề tài 1

1.2 Mục tiêu đề tài 1

1.3 Đối tượng và phạm vi nghiên cứu 2

1.4 Phương pháp thực hiện 2

1.5 Bố cục báo cáo 2

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 3

2.1 Ensemble Learning 3

2.2 Chất lượng và tính đa dạng của mô hình cơ sở 3

2.3 Cây quyết định và tiêu chí Gini 3

2.4 Bagging và lấy mẫu bootstrap 4

2.4.1 Cơ sở thống kê của bootstrap 4

2.4.2 Trực giác về phương sai và tương quan 5

2.5 Hard voting và soft voting 6

2.5.1 Hard voting 6

2.5.2 Soft voting 6

2.6 Tổng quan các hướng học tổ hợp 7

2.7 Random Forest 8

2.8 Đánh giá out-of-bag 8

2.9 Boosting: minh họa bằng AdaBoost 9

2.10 Voting với các mô hình khác loại 10

2.11 Stacking và dự đoán ngoài fold 11

CHƯƠNG 3. THUẬT TOÁN VÀ VÍ DỤ MINH HOẠ 13

3.1 Thuật toán huấn luyện Bagging 13

3.2 Thuật toán dự đoán 14

3.3 Dữ liệu minh hoạ và các mẫu bootstrap 14

3.4 Tính toán lựa chọn ngưỡng chia 15

3.4.1 Gini ban đầu của h_1 15

3.4.2 Ngưỡng 1,5 15

3.4.3 Ngưỡng 2,5 15

3.4.4 Ngưỡng 4,0 15

3.4.5 Quy tắc của ba cây 16

3.5 Dự đoán và tổng hợp phiếu 16

3.6 Minh hoạ OOB 17

3.7 Kiểm chứng ví dụ bằng Python 17

CHƯƠNG 4. BÀI TOÁN ỨNG DỤNG VÀ THIẾT KẾ THỰC NGHIỆM 19

4.1 Mô tả bài toán 19

4.2 Bộ dữ liệu 19

4.3 Chuẩn bị và chia dữ liệu 19

4.4 Lựa chọn độ sâu 20

4.5 Cấu hình mô hình đối chiếu 20

4.6 Chỉ số đánh giá 21
4.7 Thiết kế đánh giá nhiều lần chia 21
4.8 Thiết kế khảo sát số cây 22
4.9 Thiết kế đối chứng có kiểm soát 23
4.10 Thiết kế so sánh các hướng Ensemble Learning 24
CHƯƠNG 5. TRIỂN KHAI, KẾT QUẢ VÀ THẢO LUẬN 26
5.1 Môi trường và tổ chức mã nguồn 26
5.2 Triển khai bootstrap và huấn luyện 26
5.3 Triển khai hard voting 27
5.4 Triển khai OOB 28
5.5 Triển khai lựa chọn độ sâu và khảo sát số cây 29
5.5.1 Chọn độ sâu 29
5.5.2 Khảo sát số cây 29
5.5.3 Chạy thí nghiệm gốc 30
5.6 Kiểm thử chương trình 30
5.7 Kết quả lựa chọn độ sâu 31
5.8 Kết quả tại seed 42 32
5.9 Kết quả trên mười lần chia 33
5.10 Kết quả khảo sát số cây 34
5.11 Kết quả OOB 36
5.12 Phân tích trường hợp dự đoán sai 36
5.12.1 Thông tin quan sát 36
5.12.2 Đặc điểm mẫu so với train 37

5.12.3 Truy vết hai cây có dự đoán khác nhau *38*

5.12.4 Cùng điểm tổng hợp không có nghĩa cùng lỗi *39*

5.13 Đối chứng cách xét đặc trưng và quy tắc tổng hợp 39

5.13.1 Kết quả bốn cấu hình *39*

5.13.2 Chất lượng cây cơ sở và mức bất đồng *40*

5.13.3 Mã cốt lõi của phép đối chứng *41*

5.14 So sánh theo cặp trên mười lần chia gốc 41

5.15 Thảo luận chung 42

5.16 Triển khai và kết quả của AdaBoost, Voting, Stacking 42

5.16.1 Tổ chức triển khai *42*

5.16.2 Kết quả trên mười lần chia *44*

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 46

6.1 Kết quả đạt được 46

6.2 Hạn chế 46

6.3 Hướng phát triển 47

6.4 Kết luận 47

TÀI LIỆU THAM KHẢO 48

PHỤ LỤC A. HƯỚNG DẪN TÁI LẬP 49

DANH MỤC HÌNH VẼ

Hình 5.1. Ma trận nhầm lẫn của ba mô hình tại seed 42.....32

Hình 5.2. Accuracy và recall trung bình trên mười lần chia dữ liệu.....34

Hình 5.3. Ảnh hưởng số cây trong CV trên train; thanh sai số là SD giữa các fold. 35

Hình 5.4. Trường hợp bất đồng với 27 phiếu ác và 24 phiếu lành. 37

Hình 5.5. Mẫu 291 so với phân bố train; hộp Q1–Q3, vạch giữa là trung vị, nét đứt là mẫu lỗi. 38

Hình 5.6. Balanced accuracy của sáu cấu hình ensemble trên mười lần chia dữ liệu có phân tầng; thanh sai số biểu diễn độ lệch chuẩn mẫu. 44

DANH MỤC BẢNG BIỂU

Bảng 2.1. Ví dụ giả định về hard voting và soft voting 7

Bảng 2.2. Các hướng tiếp cận Ensemble Learning 7

Bảng 3.1. Dữ liệu minh hoạ 14

Bảng 3.2. Ba bootstrap 14

Bảng 3.3. So sánh ngưỡng chia của $h1$ 16

Bảng 3.4. Ngưỡng và Gini của ba cây 16

Bảng 3.5. Dự đoán tại x = 2,8 16

Bảng 3.6. Đánh giá OOB trong ví dụ 17

Bảng 4.1. Đặc điểm bộ dữ liệu 19

Bảng 4.2. Phân bố tại seed 42 20

Bảng 4.3. Các mô hình 20

Bảng 4.4. Chỉ số đánh giá 21

Bảng 4.5. Thiết kế đối chứng 2 × 2 23

Bảng 4.6. Cấu hình phần mở rộng các họ ensemble 24

Bảng 5.1. Môi trường thực nghiệm 26

Bảng 5.2. Tổ chức mã nguồn 26

Bảng 5.3. Các nhóm kiểm thử 31

Bảng 5.4. Balanced accuracy CV tại seed 42 31

Bảng 5.5. Kết quả trên tập test 32

Bảng 5.6. Số đếm phân loại 32

Bảng 5.7. Kiểm tra cách tính chỉ số Bagging 33

Bảng 5.8. Kết quả tổng hợp, trung bình ± SD mẫu 33

Bảng 5.9. Khảo sát số cây bằng CV trên train 34

Bảng 5.10. OOB của Bagging tại seed 42.....36

Bảng 5.11. Đặc điểm mẫu 291 so với hai lớp trên train 37

Bảng 5.12. Các quan sát có dự đoán khác nhau.....39

Bảng 5.13. Kết quả đối chứng trên train 39

Bảng 5.14. Chất lượng cây cơ sở và tính đa dạng.....40

Bảng 5.15. So sánh accuracy theo cặp giữa mười seed 41

Bảng 5.16. Kết quả so sánh các cấu hình trên mười holdout.....44

Bảng A.1. Các tệp kết quả phục vụ kiểm tra.....49

DANH MỤC THUẬT TOÁN

Thuật toán 3.1. Huấn luyện Bagging và tích luỹ phiếu OOB 13

Thuật toán 3.2. Dự đoán bằng hard voting.....14

Thuật toán 4.1. Khảo sát số cây trên train.....22

Thuật toán 4.2. Đối chứng đặc trưng và quy tắc tổng hợp.....24

DANH MỤC CHỮ VIẾT TẮT

Chữ viết tắt	Tên đầy đủ	Ý nghĩa
ML	Machine Learning	Học máy
CV	Cross-Validation	Đánh giá chéo
OOB	Out-of-Bag	Quan sát không có trong bootstrap của một cây
OOF	Out-of-Fold	Dự đoán ngoài fold
RF	Random Forest	Rừng ngẫu nhiên
TP	True Positive	Mẫu dương được dự đoán dương
FN	False Negative	Mẫu dương bị dự đoán âm
FP	False Positive	Mẫu âm bị dự đoán dương
TN	True Negative	Mẫu âm được dự đoán âm
SD	Standard Deviation	Độ lệch chuẩn

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Lý do thực hiện đề tài

Cây quyết định có thể thay đổi ngưỡng chia và cấu trúc khi dữ liệu huấn luyện thay đổi. Vì vậy, một cây duy nhất đôi khi cho kết quả chưa ổn định.

Ensemble Learning kết hợp nhiều mô hình. Nếu các mô hình có chất lượng phù hợp và không thường xuyên mắc cùng lỗi, việc tổng hợp có thể giảm ảnh hưởng của những sai sót riêng lẻ.

Bagging tạo nhiều tập dữ liệu bằng lấy mẫu có hoàn lại, học một mô hình trên mỗi tập rồi tổng hợp dự đoán. Nhóm chọn Bagging cây quyết định vì cơ chế rõ ràng, thuận tiện để liên hệ lý thuyết, phép tính tay và mã nguồn.

1.2 Mục tiêu đề tài

Mục tiêu tổng quát là tìm hiểu Ensemble Learning, triển khai sâu cơ chế Bagging và minh họa AdaBoost, Voting, Stacking trong một quy trình thực nghiệm có thể kiểm tra và tái lập.

Các mục tiêu cụ thể gồm giải thích nguyên lý Ensemble Learning; trình bày bootstrap, voting và OOB; xây dựng ví dụ tính tay; triển khai các bước cốt lõi của Bagging bằng Python; so sánh với cây quyết định đơn và Random Forest; phân tích lỗi và giới hạn của mô hình. Phần mở rộng minh họa và đối chiếu AdaBoost, Voting, Stacking bằng thư viện scikit-learn.

Báo cáo tập trung trả lời ba câu hỏi nghiên cứu: (1) Bagging cải thiện kết quả so với cây đơn một cách nhất quán đến mức nào trên các lần chia đã khảo sát? (2) Khi tăng số cây, chất lượng tổ hợp thay đổi ra sao và lợi ích tăng thêm có tiếp tục duy trì? (3) Khi giữ chung dữ liệu và độ sâu, cách xét đặc trưng và quy tắc tổng hợp ảnh hưởng thế nào đến chất lượng cây cơ sở, mức bất đồng và kết quả của tổ hợp?

Đóng góp của bài làm là triển khai minh bạch bootstrap, tổng hợp phiếu và OOB; xây dựng ví dụ tính tay có thể kiểm tra; tổ chức các đối chiếu thực nghiệm có kiểm soát; và minh họa sự khác nhau giữa các cơ chế học tổ hợp. Các phương pháp

nền tảng dựa trên tài liệu đã công bố; kết luận thực nghiệm được giới hạn trong dữ liệu và giao thức của báo cáo.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là Ensemble Learning cho bài toán phân loại, trong đó Bagging cây quyết định là trọng tâm triển khai và tính toán minh họa. AdaBoost, Voting và Stacking được bổ sung để đối chiếu các cơ chế tổ hợp.

Phần ứng dụng sử dụng bộ dữ liệu Breast Cancer Wisconsin Diagnostic. Đầu vào là các đặc trưng số đã trích xuất; đầu ra là nhãn ác tính hoặc lành tính.

Cây quyết định cơ sở và Random Forest sử dụng scikit-learn. Cơ chế lấy mẫu bootstrap, đếm phiếu và tính OOB được triển khai trong lớp ManualBagging.

AdaBoost, Voting hard/soft và Stacking được triển khai minh họa bằng scikit-learn trên cùng các lần chia dữ liệu. Các cấu hình được công bố ở mục 4.10. Bài làm mang mục đích học thuật, chưa đánh giá khả năng sử dụng trong chẩn đoán lâm sàng.

1.4 Phương pháp thực hiện

Phương pháp thực hiện gồm tìm hiểu các tài liệu nền tảng; xây dựng và kiểm chứng ví dụ nhỏ; triển khai mô hình với lựa chọn cấu hình chỉ trên train; sau đó tổng hợp kết quả, khảo sát số cây, thực hiện đối chứng và truy vết mẫu dự đoán sai.

1.5 Bố cục báo cáo

Chương 2 trình bày cơ sở lý thuyết. Chương 3 mô tả thuật toán và ví dụ minh họa. Chương 4 giới thiệu bài toán ứng dụng cùng thiết kế thực nghiệm. Chương 5 giải thích mã nguồn và phân tích kết quả. Chương 6 trình bày kết luận, hạn chế và hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Ensemble Learning

Ensemble Learning là phương pháp kết hợp nhiều mô hình cơ sở để tạo ra một mô hình tổng hợp. Trong phân loại, các mô hình cơ sở có thể cung cấp nhãn hoặc xác suất dự đoán. Bộ kết hợp sử dụng những đầu ra này để chọn nhãn cuối cùng.

Ví dụ, ba mô hình lần lượt dự đoán A, B và A. Nếu dùng bỏ phiếu đa số, tổ hợp chọn A vì lớp này nhận hai trong ba phiếu.

Tuy nhiên, nhiều mô hình không tự động bảo đảm kết quả tốt. Nếu các mô hình cùng sai trên những quan sát giống nhau, việc kết hợp có thể giữ nguyên các lỗi đó [1], [2].

2.2 Chất lượng và tính đa dạng của mô hình cơ sở

Một tổ hợp cần các mô hình cơ sở học được thông tin hữu ích từ dữ liệu. Đồng thời, các mô hình nên có sự khác nhau nhất định về dự đoán và cấu trúc lỗi.

Sự khác nhau có thể được tạo bằng cách thay đổi dữ liệu huấn luyện, đặc trưng hoặc thuật toán. Trong Bagging, nguồn tạo đa dạng chính là các tập bootstrap.

Cây quyết định nhạy với thay đổi dữ liệu. Hai bootstrap khác nhau có thể tạo những ngưỡng chia khác nhau. Khi tổng hợp nhiều cây, ảnh hưởng của một cây sai riêng lẻ có thể được giảm bớt.

Đây là trực giác về khả năng giảm phương sai của Bagging. Trong phạm vi nhập môn, có thể hiểu phương sai là mức độ kết quả mô hình thay đổi khi tập huấn luyện thay đổi. Khả năng giảm dao động vẫn cần được kiểm tra bằng thực nghiệm.

2.3 Cây quyết định và tiêu chí Gini

Cây quyết định phân loại một quan sát bằng chuỗi điều kiện trên các đặc trưng. Tại mỗi nút, quan sát đi theo nhánh phù hợp và cuối cùng nhận nhãn tại lá.

Độ hỗn tạp Gini của một nút được tính:

$$\text{Gini} = 1 - \sum_{c=1}^{C} p_c^2. \quad (2.1)$$

Trong đó C là số lớp và p_c là tỷ lệ quan sát thuộc lớp c tại nút.

Với hai lớp:

$$\text{Gini} = 1 - \text{p}_{\text{ác}}^2 - \text{p}_{\text{lành}}^2.$$

Nút chỉ chứa một lớp có Gini bằng 0. Sau một phép chia:

$$\text{Gini}_{\text{sau chia}} = \frac{\text{N}_{\text{trái}}}{\text{N}} \times \text{Gini}_{\text{trái}} + \frac{\text{N}_{\text{phải}}}{\text{N}} \times \text{Gini}_{\text{phải}}$$

Mức giảm Gini:

$$\Delta_{\text{Gini}} = \text{Gini}_{\text{trước chia}} - \text{Gini}_{\text{sau chia}}$$

Trong các phép chia hợp lệ được xét, cây ưu tiên phép chia tạo mức giảm lớn hơn [5].

2.4 Bagging và lấy mẫu bootstrap

Bagging là viết tắt của **bootstrap aggregating** [1].

Cho tập train có N quan sát, một bootstrap được tạo bằng N lượt lấy mẫu có hoàn lại. Một quan sát vừa được lấy vẫn có thể được lấy tiếp. Do đó:

- Một quan sát có thể xuất hiện nhiều lần.
- Một quan sát có thể không xuất hiện.
- Tổng số lượt lấy vẫn bằng N.
- Số quan sát khác nhau có thể nhỏ hơn N.

Ví dụ, từ các chỉ số [0, 1, 2, 3, 4], bootstrap [0, 0, 2, 4, 1] có chỉ số 0 lặp hai lần và thiếu chỉ số 3.

Bagging tạo B bootstrap và huấn luyện một mô hình trên mỗi mẫu. Các mô hình học độc lập về mặt thuật toán, sau đó cùng dự đoán một quan sát mới.

Bagging không chia train thành B phần rời nhau. Mỗi bootstrap đều được lấy từ cùng tập train ban đầu.

2.4.1 Cơ sở thống kê của bootstrap

Với N quan sát, xác suất một quan sát cố định không được chọn trong một lượt là:

$$1 - \frac{1}{N}.$$

Do bootstrap gồm N lượt lấy có hoàn lại độc lập, xác suất quan sát đó không xuất hiện trong bootstrap là:

$$q = \left(1 - \frac{1}{N}\right)^N$$

Gọi I_i bằng 1 nếu quan sát i xuất hiện trong bootstrap và bằng 0 nếu không. Số quan sát khác nhau là

$$U = \sum_{i=1}^{N} I_i.$$

Theo tính tuyến tính của kỳ vọng:

$$\mathbb{E}[U] = \sum_{i=1}^{N} \mathbb{E}\,[I_i] = \sum_{i=1}^{N} P\,(I_i = 1) = N\left[1 - \left(1 - \frac{1}{N}\right)^N\right]. \quad (2.2)$$

Với N = 426, số quan sát khác nhau kỳ vọng là 269,47, tương ứng 63,26% tập train; số quan sát OOB kỳ vọng mỗi cây là 156,53. Với $B = 51$, mỗi quan sát train kỳ vọng nhận $Bq \approx 18{,}74$ phiếu OOB. Đây là kỳ vọng của các đại lượng ngẫu nhiên, không phải số lượng cố định trong mọi lượt chạy.

Trong 51 bootstrap thực của mô hình tham chiếu seed 42, số quan sát khác nhau trung bình là 268,04, nhỏ nhất 254 và lớn nhất 281. Số phiếu OOB trung bình trên mỗi quan sát là 18,91. Các con số giúp kiểm chứng cách lấy mẫu và làm rõ rằng 426 lượt lấy không đồng nghĩa với 426 quan sát khác nhau.

2.4.2 Trực giác về phương sai và tương quan

Nếu các dự đoán số có cùng phương sai σ^2 và cùng tương quan cặp ρ, phương sai của trung bình B dự đoán là:

$$\mathrm{Var}(\bar{h}) = \sigma^2\left[\rho + \frac{1 - \rho}{B}\right]. \quad (2.3)$$

Khi B tăng, thành phần:

$$\frac{1 - \rho}{B}$$

giảm nhưng thành phần ρ vẫn còn. Với $B = 51, \rho = 0{,}2$ cho khoảng $0{,}216\sigma^2$, còn $\rho = 0{,}8$ cho khoảng $0{,}804\sigma^2$. Vì vậy, tăng số mô hình quá giống nhau có thể mang lại lợi ích hạn chế [1], [2].

Công thức cung cấp trực giác cho trung bình dự đoán số dưới các giả thiết đã nêu; đây không phải công thức trực tiếp tính lỗi phân loại hard voting. Các giá trị ρ chỉ dùng minh họa, không được suy từ mức bất đồng nhãn trong thực nghiệm.

2.5 Hard voting và soft voting

2.5.1 Hard voting

Hard voting đếm số mô hình chọn mỗi lớp và lấy lớp có nhiều phiếu nhất:

$$H(x) = \arg\max_{c} \sum_{b=1}^{B} I\left[h_b(x) = c\right].$$

Trong đó:

- $H(x)$: nhãn của tổ hợp.
- $h_b(x)$: nhãn do mô hình thứ b dự đoán.
- I: hàm chỉ thị, bằng 1 khi điều kiện đúng và bằng 0 khi sai.
- B: số mô hình cơ sở.
- c: nhãn lớp đang xét.

Khi có hai lớp và 51 cây, mỗi cây bỏ một phiếu nên không có hòa phiếu. Với nhiều hơn hai lớp, số cây lẻ vẫn có thể tạo hòa. Mã nguồn quy định khi hòa sẽ chọn lớp đầu theo classes_.

2.5.2 Soft voting

Soft voting lấy trung bình xác suất dự đoán của từng lớp rồi chọn lớp có giá trị trung bình lớn nhất [3].

Xét ví dụ giả định:

Bảng 2.1. Ví dụ giả định về hard voting và soft voting

Mô hình	Xác suất dự đoán ác tính	Nhãn riêng
1	0,51	Ác
2	0,51	Ác
3	0,01	Lành

Hard voting chọn ác vì có hai phiếu ác và một phiếu lành.

Soft voting tính:

$$P_{\text{trung bình của lớp ác}} = \frac{0{,}51 + 0{,}51 + 0{,}01}{3} = 0{,}343$$

Xác suất lớp lành trung bình khoảng 0,657, nên soft voting chọn lành.

Ví dụ cho thấy quy tắc tổng hợp có thể làm thay đổi kết quả. Soft voting không tự động tốt hơn hard voting; hiệu quả còn phụ thuộc chất lượng các xác suất đầu vào.

2.6 Tổng quan các hướng học tổ hợp

Bảng 2.2. Các hướng tiếp cận Ensemble Learning

Phương pháp	Ý tưởng chính	Vai trò trong báo cáo
Bagging	Học trên các mẫu bootstrap rồi tổng hợp dự đoán	Trọng tâm tự triển khai và tính tay
Boosting	Học tuần tự để xử lý sai sót hoặc giảm hàm mất mát	Minh họa bằng AdaBoost
Voting	Kết hợp nhãn hoặc xác suất dự đoán	Quy tắc kết hợp và mô hình dị loại
Stacking	Học mô hình tầng hai từ dự đoán của các mô hình cơ sở	Minh họa OOF và tầng kết hợp

Với AdaBoost, mẫu dự đoán sai được tăng trọng số tương đối để mô hình tiếp theo chú ý hơn [6]. Gradient Boosting xây dựng mô hình theo hướng giảm hàm mất mát. Vì vậy, không nên dùng riêng cách cập nhật trọng số của AdaBoost để mô tả mọi phương pháp Boosting.

Với Stacking, dự đoán của các mô hình cơ sở trở thành đầu vào cho mô hình tầng hai. Khi huấn luyện tầng hai, cần dùng dự đoán ngoài fold để mỗi quan sát được dự đoán bởi mô hình cơ sở chưa học quan sát đó, qua đó hạn chế rò rỉ dữ liệu [7].

Các cách tiếp cận không hoàn toàn tách rời. Bagging vẫn cần một quy tắc tổng hợp như voting.

2.7 Random Forest

Random Forest kết hợp nhiều cây, thường sử dụng bootstrap cùng ngẫu nhiên hóa việc chọn đặc trưng tại từng phép chia [2].

Trong cấu hình của báo cáo, RandomForestClassifier dùng max_features='sqrt'. Với 30 đặc trưng, số đặc trưng mục tiêu tại một phép chia là 5. Thư viện có thể xem thêm đặc trưng khi cần để tìm phép chia hợp lệ.

Trong scikit-learn, Random Forest tổng hợp bằng trung bình xác suất lớp của các cây. ManualBagging trong bài sử dụng đa số nhãn. Vì vậy, đối chiếu hai mô hình khác nhau ở cả cách chọn đặc trưng và cách tổng hợp.

2.8 Đánh giá out-of-bag

Một quan sát không xuất hiện trong bootstrap của một cây được gọi là OOB đối với cây đó.

Khi đánh giá OOB cho một quan sát train, chỉ những cây chưa học quan sát đó mới được bỏ phiếu. Các phiếu hợp lệ được tổng hợp thành dự đoán OOB.

Với N quan sát:

$$\mathrm{P}(\text{một quan sát không được chọn}) = \left(1 - \frac{1}{\mathrm{N}}\right)^{\mathrm{N}}$$

Khi N lớn, xác suất tiến gần 36,8%.

Hai đại lượng cần theo dõi:

$$\text{Coverage OOB} = \frac{\text{Số quan sát có ít nhất một phiếu OOB}}{\mathrm{N}}$$

$$\text{Accuracy OOB} = \frac{\text{Số dự đoán OOB đúng}}{\text{Số quan sát có phiếu OOB}}$$

Mẫu không có phiếu phải được giữ ở trạng thái không xác định, thay vì tự động gán một lớp.

2.9 Boosting: minh họa bằng AdaBoost

AdaBoost xây dựng tuần tự các mô hình cơ sở. Mỗi vòng học sử dụng trọng số của các quan sát; những mẫu bị dự đoán sai được tăng trọng số tương đối để mô hình ở vòng tiếp theo chú ý hơn. Phần minh họa sử dụng cây quyết định độ sâu 1, còn gọi là decision stump, với tối đa 51 vòng và learning_rate bằng 1 [3], [6].

Trong bài toán hai lớp, quy trình SAMME được thực hiện như sau:

1. Khởi tạo trọng số bằng nhau cho các quan sát: $w_i = \frac{1}{N}$.
2. Huấn luyện cây h_t bằng các trọng số hiện tại.
3. Tính sai số có trọng số ϵ_t.
4. Tính trọng số phiếu α_t của cây.
5. Tăng trọng số tương đối của các mẫu bị dự đoán sai và chuẩn hóa tổng trọng số về 1.
6. Lặp lại quá trình, sau đó tổng hợp dự đoán bằng phiếu có trọng số.

Với trọng số quan sát đã được chuẩn hóa, sai số có trọng số được tính:

$$\epsilon_t = \sum_{i=1}^{N} w_i \, I[h_t(x_i) \neq y_i].$$

Trong đó, $(I[\cdot])$ là hàm chỉ báo, nhận giá trị 1 khi điều kiện đúng và 0 khi điều kiện sai.

Với $(0 < \epsilon_t < 0{,}5)$ và learning_rate bằng 1, trọng số của cây là:

$$\alpha_{\mathrm{t}} = ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right).$$

Trọng số quan sát được cập nhật theo:

$$w_i^{\text{mới}} \propto w_i \exp(\alpha_t I[h_t(x_i) \neq y_i]).$$

Sau khi cập nhật, các trọng số được chia cho tổng của chúng để bảo đảm tổng trọng số bằng 1. Nhãn dự đoán cuối cùng là lớp nhận tổng trọng số phiếu lớn nhất:

$$H(x) = \arg\max_{c} \sum_{t} \alpha_t \, I[h_t(x) = c].$$

Đây là quy ước SAMME của AdaBoostClassifier trong scikit-learn 1.8 cho hai lớp, không dùng trực tiếp hệ số ½ của một cách viết AdaBoost nhị phân khác. Cây hoàn hảo có thể làm thuật toán dừng sớm; vì vậy 51 là số vòng tối đa, không luôn là số cây thực học.

Ví dụ, xét năm quan sát có trọng số ban đầu bằng nhau, mỗi quan sát nhận trọng số 0,2. Giả sử cây đầu tiên dự đoán sai một quan sát, khi đó:

$$\epsilon = 0{,}2, \qquad \alpha = \ln\left(\frac{0{,}8}{0{,}2}\right) = \ln 4 \approx 1{,}386.$$

Trọng số của mẫu sai được nhân với:

$$\exp(\alpha) = 4.$$

Do đó, mẫu sai có trọng số chưa chuẩn hóa là $(0{,}2 \times 4 = 0{,}8)$. Bốn mẫu đúng giữ trọng số 0,2 mỗi mẫu. Tổng trọng số chưa chuẩn hóa bằng:

$$0{,}8 + 4 \times 0{,}2 = 1{,}6.$$

Sau chuẩn hóa, mẫu sai nhận trọng số:

$$\frac{0{,}8}{1{,}6} = 0{,}5,$$

và mỗi mẫu đúng nhận trọng số:

$$\frac{0{,}2}{1{,}6} = 0{,}125.$$

Ví dụ cho thấy vòng học tiếp theo tập trung hơn vào mẫu bị dự đoán sai. Giá trị 50% ở đây là tỷ trọng của mẫu sai trong tổng trọng số, không phải accuracy đã tăng lên 50%.

2.10 Voting với các mô hình khác loại

Voting trong phần mở rộng kết hợp ba mô hình khác loại: cây quyết định độ sâu 3, Logistic Regression và Gaussian Naive Bayes. Cây quyết định chia không gian đặc trưng bằng các điều kiện; Logistic Regression học ranh giới phân loại tuyến tính; Gaussian NB mô hình hóa phân bố đặc trưng theo lớp. Sự khác nhau về giả thiết học

có thể tạo ra các dự đoán bổ sung cho nhau, nhưng không bảo đảm tổ hợp luôn tốt hơn từng mô hình.

Quy trình thực hiện gồm huấn luyện riêng ba mô hình trên tập train; lấy nhãn dự đoán cho hard voting hoặc xác suất lớp cho soft voting; tổng hợp với trọng số bằng nhau; sau đó đánh giá trên tập test.

Với hard voting, lớp nhận nhiều phiếu nhất được chọn làm nhãn cuối cùng. Với soft voting, xác suất của từng lớp được lấy trung bình giữa các mô hình, sau đó chọn lớp có xác suất trung bình lớn nhất. Các công thức tại mục 2.5 tiếp tục được áp dụng.

Hai biến thể hard và soft sử dụng cùng cấu hình mô hình cơ sở để việc đối chiếu không đồng thời thay đổi cả kiến trúc và quy tắc kết hợp. Soft voting có thể chịu ảnh hưởng bởi chất lượng xác suất của từng mô hình; báo cáo chưa thực hiện hiệu chuẩn xác suất [3].

2.11 Stacking và dự đoán ngoài fold

Stacking học một mô hình tầng hai để kết hợp đầu ra của các mô hình cơ sở [7]. Ba mô hình cơ sở giống phần Voting; mô hình tầng hai là Logistic Regression. Khác với Voting sử dụng quy tắc kết hợp cố định, Stacking học cách kết hợp từ dữ liệu.

Quy trình trên tập train được thực hiện như sau:

1. Chia tập train thành năm fold phân tầng.
2. Trong từng fold, huấn luyện các mô hình cơ sở trên bốn fold còn lại và dự đoán xác suất cho fold được giữ.
3. Ghép các dự đoán ngoài fold, gọi là out-of-fold hoặc OOF, thành dữ liệu đầu vào cho tầng hai.
4. Huấn luyện mô hình tầng hai bằng các đầu ra OOF và nhãn tương ứng của tập train.
5. Huấn luyện lại các mô hình cơ sở trên toàn bộ tập train để sử dụng khi dự đoán dữ liệu mới.

Khi gặp tập test, ba mô hình cơ sở tạo ra các đầu ra xác suất. Những đầu ra này trở thành đầu vào cho mô hình tầng hai để dự đoán nhãn cuối cùng.

Trong scikit-learn, với bài toán hai lớp và stack_method bằng predict_proba, một cột xác suất dư thừa được bỏ ở mỗi mô hình cơ sở. Ba cột còn lại tương ứng với xác suất lớp 1, tức lớp lành tính, không mặc định là xác suất ác tính.

StandardScaler nằm trong Pipeline của Logistic Regression nên mỗi lần fit chỉ học thống kê từ phần dữ liệu được phép sử dụng. Trong quá trình tạo OOF, scaler được học lại trên phần train của từng fold. Tập test không tham gia huấn luyện mô hình cơ sở hoặc mô hình tầng hai.

Việc sử dụng OOF giúp mỗi quan sát train nhận dự đoán từ mô hình chưa học quan sát đó. Không sử dụng dự đoán in-sample để thay thế OOF khi huấn luyện tầng hai [3], [7].

CHƯƠNG 3. THUẬT TOÁN VÀ VÍ DỤ MINH HOẠ

3.1 Thuật toán huấn luyện Bagging

Thuật toán nhận tập train, số cây B và độ sâu tối đa d. Độ sâu được lựa chọn bằng quy trình CV trên train trước khi huấn luyện mô hình cuối.

Thuật toán 3.1. Huấn luyện Bagging và tích luỹ phiếu OOB

```
Đầu vào:
   X, y: N quan sát huấn luyện và nhãn
   B: số cây
   d: độ sâu tối đa
   seed: hạt giống ngẫu nhiên

Đầu ra:
   Danh sách cây T
   Thứ tự các lớp C
   Ma trận phiếu OOB V

1. Xác định các lớp C và mã hóa nhãn thành chỉ số liên tiếp.
2. Khởi tạo bộ sinh ngẫu nhiên theo seed.
3. Khởi tạo T rỗng và V = 0, kích thước N × số lớp.
4. Với b từ 1 đến B:
      a. Lấy N chỉ số có hoàn lại từ {0, ..., N−1}.
      b. Tạo bootstrap từ các chỉ số vừa lấy.
      c. Huấn luyện cây độ sâu tối đa d trên bootstrap.
      d. Lưu cây vào T.
      e. Xác định các chỉ số không xuất hiện trong bootstrap.
      f. Dùng cây dự đoán các quan sát chưa xuất hiện.
      g. Cộng phiếu của cây vào V cho các quan sát đó.
5. Tính coverage từ các hàng V có ít nhất một phiếu.
6. Tính accuracy OOB chỉ trên các hàng có phiếu.
7. Trả về T, C và các kết quả OOB.
```

Các cây học từ nhãn mã hóa nên quá trình đếm phiếu nhất quán ngay cả khi nhãn gốc là chuỗi hoặc các số không liên tiếp.

3.2 Thuật toán dự đoán

Thuật toán 3.2. Dự đoán bằng hard voting

```
Đầu vào:
   T: danh sách B cây đã học
   C: thứ tự các lớp
   X_new: các quan sát cần dự đoán

Đầu ra:
   Nhãn dự đoán của từng quan sát

1. Khởi tạo ma trận đếm phiếu bằng 0.
2. Với mỗi cây trong T:
      a. Dự đoán nhãn cho X  new.
      b. Tăng một phiếu ở cột lớp tương ứng của từng quan sát.
3. Với mỗi quan sát:
      a. Tìm cột có số phiếu lớn nhất.
      b. Nếu hòa, chọn cột đầu theo thứ tự C.
      c. Ánh xạ chỉ số cột về nhãn gốc.
4. Trả về các nhãn.
```

Mỗi quan sát nhận đúng B phiếu. Với K quan sát mới và C lớp, ma trận phiếu có kích thước K × C.

3.3 Dữ liệu minh hoạ và các mẫu bootstrap

Sử dụng năm quan sát giả lập một chiều

Bảng 3.1. Dữ liệu minh hoạ

Chỉ số	Giá trị x	Nhãn
0	1	Lành — 1
1	2	Lành — 1
2	3	Ác — 0
3	4	Ác — 0
4	5	Ác — 0

Giá trị x không đại diện cho một đặc trưng y tế cụ thể. Các cây có độ sâu tối đa bằng 1 để phép tính dễ theo dõi.

Bảng 3.2. Ba bootstrap

Cây	Chỉ số bootstrap	Giá trị x tương ứng	Hàng không có mặt
h_1	[0, 0, 2, 4, 1]	[1, 1, 3, 5, 2]	3
h_2	[0, 0, 1, 3, 4]	[1, 1, 2, 4, 5]	2
h_3	[4, 2, 4, 1, 3]	[5, 3, 5, 2, 4]	0

3.4 Tính toán lựa chọn ngưỡng chia

3.4.1 Gini ban đầu của h_1

Bootstrap h_1 gồm ba mẫu lành và hai mẫu ác:

$$\text{Gini}_{\text{ban đầu}} = 1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 0{,}48$$

Các giá trị x khác nhau là 1, 2, 3 và 5. Các ngưỡng ứng viên nằm giữa hai giá trị liên tiếp là 1,5; 2,5 và 4,0.

3.4.2 Ngưỡng 1,5

Nhánh trái có hai mẫu lành nên Gini bằng 0.

Nhánh phải có một mẫu lành và hai mẫu ác:

$$\text{Gini}_{\text{phải}} = 1 - \left(\frac{1}{3}\right)^2 - \left(\frac{2}{3}\right)^2 = \frac{4}{9}$$

Gini sau chia:

$$\frac{2}{5} \times 0 + \frac{3}{5} \times \frac{4}{9} \approx 0{,}2667$$

Mức giảm:

$$0{,}48 - 0{,}2667 \approx 0{,}2133$$

3.4.3 Ngưỡng 2,5

Nhánh trái có ba mẫu lành; nhánh phải có hai mẫu ác. Cả hai nhánh thuần nhất:

$$\text{Gini}_{\text{sau chia}} = \frac{3}{5} \times 0 + \frac{2}{5} \times 0 = 0$$

Mức giảm bằng 0,48.

3.4.4 Ngưỡng 4,0

Nhánh trái có ba mẫu lành và một mẫu ác:

$$\text{Gini}_{\text{trái}} = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = 0{,}375$$

Nhánh phải có một mẫu ác nên Gini bằng 0.

$$\text{Gini}_{\text{sau chia}} = \frac{4}{5} \times 0{,}375 + \frac{1}{5} \times 0 = 0{,}30.$$

Mức giảm bằng 0,18.

Bảng 3.3. So sánh ngưỡng chia của h_1

Ngưỡng	Gini sau chia	Mức giảm Gini
1,5	0,2667	0,2133
2,5	0	0,48
4,0	0,30	0,18

Ngưỡng 2,5 tạo mức giảm lớn nhất nên được chọn.

3.4.5 Quy tắc của ba cây

Bảng 3.4. Ngưỡng và Gini của ba cây

Cây	Thành phần bootstrap	Ngưỡng	Gini trước → sau
h_1	3 lành, 2 ác	2,5	0,48 → 0
h_2	3 lành, 2 ác	3,0	0,48 → 0
h_3	1 lành, 4 ác	2,5	0,32 → 0

Bootstrap h_2 không chứa x = 3. Hai giá trị sát ranh giới lớp là x = 2 và x = 4 nên cây chọn ngưỡng 3,0. Hai cây còn lại có x = 2 và x = 3 nên ngưỡng là 2,5.

Sự thay đổi dữ liệu bootstrap đã tạo ra ranh giới khác nhau giữa các cây.

3.5 Dự đoán và tổng hợp phiếu

Đưa cùng quan sát mới x = 2,8 qua ba cây.

Bảng 3.5. Dự đoán tại x = 2,8

Cây	So sánh	Dự đoán
h_1	2,8 > 2,5	Ác — 0
h_2	2,8 ≤ 3,0	Lành — 1
h_3	2,8 > 2,5	Ác — 0

Có hai phiếu ác và một phiếu lành, nên tổ hợp chọn ác.

Quan sát mới chưa có nhãn thật nên chưa kết luận dự đoán đúng hay sai. Tỷ lệ 2/3 chỉ là tỷ lệ phiếu, không phải xác suất bệnh đã hiệu chỉnh.

3.6 Minh hoạ OOB

Bảng 3.6. Đánh giá OOB trong ví dụ

Quan sát	**Cây đủ điều kiện OOB**	**Thật → dự đoán**	**Kết quả**
Hàng 0, x = 1	h_3	Lành → Lành	Đúng
Hàng 2, x = 3	h_2	Ác → Lành	Sai
Hàng 3, x = 4	h_1	Ác → Ác	Đúng
Hàng 1 và 4	Không có	Không xác định	Không tính accuracy

Ba trong năm quan sát có phiếu:

$$\text{Coverage OOB} = \frac{3}{5} = 60\%.$$

Hai trong ba dự đoán hợp lệ đúng:

$$\text{Accuracy OOB} = \frac{2}{3} \approx 66{,}67\%$$

Hàng 2 chỉ nhận phiếu từ h_2 vì h_1 và h_3 đã học hàng đó. Cây h_2 dự đoán lành tại x = 3 do điều kiện $x \leq 3$, tạo thành một lỗi OOB.

Với N = 5:

$$\text{P(không được chọn)} = \left(\frac{4}{5}\right)^5 = 32{,}768\%$$

Giá trị này khác giới hạn xấp xỉ 36,8% khi N lớn.

3.7 Kiểm chứng ví dụ bằng Python

Đoạn mã sau có thể chạy độc lập sau khi cài NumPy và scikit-learn:

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier

X = np.arange(1, 6, dtype=float).reshape(-1, 1)
y = np.array([1, 1, 0, 0, 0])

bootstraps = [
    [0, 0, 2, 4, 1],
    [0, 0, 1, 3, 4],
    [4, 2, 4, 1, 3],
]

predictions = []

for number, indices in enumerate(bootstraps, start=1):
    tree = DecisionTreeClassifier(
        max_depth=1,
        random_state=number,
    )
    tree.fit(X[indices], y[indices])

    prediction = int(tree.predict([[2.8]])[0])
    predictions.append(prediction)

    print(
        f"h{number}: threshold={tree.tree_.threshold[0]}, "
        f"prediction={prediction}"
    )

votes = np.bincount(predictions, minlength=2)

print("Votes [malignant, benign]:", votes)
print("Ensemble prediction:", votes.argmax())
```

Kết quả tương ứng:

```
h1: threshold=2.5, prediction=0
h2: threshold=3.0, prediction=1
h3: threshold=2.5, prediction=0
Votes [malignant, benign]: [2 1]
Ensemble prediction: 0
```

CHƯƠNG 4. BÀI TOÁN ỨNG DỤNG VÀ THIẾT KẾ THỰC NGHIỆM

4.1 Mô tả bài toán

Bài toán là phân loại mẫu khối u vú thành ác tính hoặc lành tính dựa trên các đặc trưng đã trích xuất.

Đây là bài toán phân loại có giám sát. Mỗi quan sát train gồm vector đặc trưng và nhãn. Mô hình học từ những cặp dữ liệu này để dự đoán nhãn của quan sát mới.

Bài toán cho phép phân tích những loại lỗi có ý nghĩa khác nhau: mẫu ác bị dự đoán lành và mẫu lành bị dự đoán ác. Vì vậy, đánh giá sử dụng nhiều chỉ số thay vì chỉ accuracy.

4.2 Bộ dữ liệu

Dữ liệu được nạp bằng load_breast_cancer của scikit-learn [4].

Bảng 4.1. Đặc điểm bộ dữ liệu

Thuộc tính	**Mô tả**
Tên	Breast Cancer Wisconsin Diagnostic
Số quan sát	569
Số đặc trưng	30 đặc trưng số
Nhãn 0	Malignant — ác tính
Nhãn 1	Benign — lành tính
Số mẫu ác	212
Số mẫu lành	357

Các đặc trưng mô tả tính chất nhân tế bào, chẳng hạn bán kính, kết cấu, chu vi và diện tích. Đầu vào chương trình là vector 30 giá trị, không phải ảnh gốc.

4.3 Chuẩn bị và chia dữ liệu

Ba mô hình trong thí nghiệm gốc đều dựa trên cây nên không yêu cầu chuẩn hóa thang đo như một số mô hình sử dụng khoảng cách. Đối với phần mở rộng, Logistic Regression được kết hợp với StandardScaler trong Pipeline; các bước chuẩn

hóa chỉ được học từ phần dữ liệu huấn luyện tương ứng. Mỗi seed chia dữ liệu theo tỷ lệ 75% train và 25% test, có phân tầng theo nhãn.

Bảng 4.2. Phân bố tại seed 42

Tập	Tổng số mẫu	Ác tính	Lành tính
Train	426	159	267
Test	143	53	90

Tỷ lệ 75/25 dành phần lớn dữ liệu cho huấn luyện và giữ một phần để đánh giá. Đây là lựa chọn thực nghiệm, chưa được xác định là tỷ lệ tối ưu.

Test không tham gia lựa chọn độ sâu hay số cây.

4.4 Lựa chọn độ sâu

Độ sâu ảnh hưởng đến độ phức tạp của cây. Cây quá nông có thể chưa học đủ quan hệ; cây quá sâu có thể bám sát những đặc điểm riêng của train.

Các độ sâu ứng viên là:

$$d \in \{2,3,4,5,6\}$$

Chương trình sử dụng CV phân tầng năm fold trên train. Với mỗi độ sâu, cây học trên bốn fold và được chấm trên fold còn lại. Điểm balanced accuracy trung bình được dùng để lựa chọn.

Nếu hòa điểm, độ sâu nhỏ hơn được ưu tiên. Sau đó, mô hình được huấn luyện trên toàn train trước khi đánh giá test.

4.5 Cấu hình mô hình đối chiếu

Bảng 4.3. Các mô hình

Mô hình	Cấu hình	Tổng hợp
Cây đơn	Một cây với độ sâu chọn bằng CV	Dự đoán trực tiếp
Manual Bagging	51 cây trên bootstrap	Đa số nhãn
Random Forest	51 cây, max_features='sqrt'	Trung bình xác suất

Độ sâu được chọn bằng cây đơn rồi dùng chung cho ba mô hình. Thiết kế này thuận tiện cho minh họa nhưng chưa tối ưu riêng từng ensemble.

Số cây 51 là cấu hình ban đầu cố định và tránh hòa phiếu trong bài toán hai lớp. Khảo sát số cây được thực hiện bổ sung sau đó, không được dùng để viết lại lịch sử lựa chọn cấu hình ban đầu.

4.6 Chỉ số đánh giá

Quy ước ác tính, nhãn 0, là lớp dương:

- TP: mẫu ác được dự đoán ác.
- FN: mẫu ác bị dự đoán lành.
- FP: mẫu lành bị dự đoán ác.
- TN: mẫu lành được dự đoán lành.

Bảng 4.4. Chỉ số đánh giá

Chỉ số	**Công thức**	**Ý nghĩa**
Accuracy	(TP + TN)/N	Tỷ lệ đúng tổng thể
Precision ác	TP/(TP + FP)	Tỷ lệ ác thật trong các dự đoán ác
Recall ác	TP/(TP + FN)	Tỷ lệ mẫu ác được phát hiện
F1 ác	2TP/(2TP + FP + FN)	Trung bình điều hòa precision và recall
Specificity	TN/(TN + FP)	Tỷ lệ mẫu lành được nhận diện đúng
Balanced accuracy	(Recall ác + Recall lành)/2	Trung bình recall hai lớp

Recall và FN phản ánh khả năng phát hiện mẫu ác. Đồng thời, precision và FP giúp đánh giá sự đánh đổi khi mô hình dự đoán ác tính.

Do lớp dương có nhãn 0, mã nguồn đặt pos_label=0 khi tính precision, recall và F1.

4.7 Thiết kế đánh giá nhiều lần chia

Thí nghiệm gốc sử dụng các seed từ 42 đến 51. Mỗi seed thực hiện lại:

1. Chia train/test có phân tầng.
2. Chọn độ sâu bằng CV trên train.
3. Huấn luyện ba mô hình.
4. Đánh giá trên cùng tập test của seed đó.

5. Lưu kết quả để tổng hợp.

Seed 42 được giữ làm lần tham chiếu để phân tích chi tiết. Trung bình và SD mẫu trên mười seed mô tả mức kết quả chung cùng độ dao động.

SD được tính với mẫu số $n - 1$, tương ứng ddof=1. Các tập test giữa các seed có thể giao nhau, nên SD chỉ là thống kê mô tả, không phải khoảng tin cậy độc lập.

4.8 Thiết kế khảo sát số cây

Khảo sát bổ sung xét:

$$B \in \{1, 5, 11, 21, 51, 101\}$$

Chỉ 426 mẫu train của seed 42 được sử dụng. Tập test 143 mẫu không tham gia khảo sát.

Train được chia thành năm fold ngoài có phân tầng. Trong phần học của mỗi fold ngoài, CV năm fold bên trong chọn độ sâu bằng cây đơn. Sau đó, các cấu hình B được huấn luyện và chấm trên fold ngoài còn lại.

Thuật toán 4.1. Khảo sát số cây trên train

```
Đầu vào:
   X_train, y_train
   Các số cây {1, 5, 11, 21, 51, 101}

Đầu ra:
   Điểm CV của từng cấu hình

1. Chia train thành 5 fold ngoài có phân tầng.
2. Với mỗi fold ngoài:
      a. Giữ một fold làm tập đánh giá.
      b. Chỉ dùng phần học để chọn độ sâu bằng CV 5 fold.
      c. Với từng số cây B:
            - Học Bagging trên phần học.
            - Đánh giá trên fold ngoài được giữ lại.
            - Lưu chỉ số.
3. Tính trung bình và SD giữa 5 fold cho mỗi B.
```

Trong cùng fold ngoài, các B dùng chung độ sâu và chuỗi seed tương ứng. Độ sâu chọn được trong năm fold lần lượt là 2, 6, 2, 2 và 6.

Điểm cực đại của bảng khảo sát không được coi là một đánh giá độc lập cho quy trình đã lựa chọn B. Báo cáo giữ nguyên các kết quả test của thí nghiệm gốc.

4.9 Thiết kế đối chứng có kiểm soát

Đối chiếu trực tiếp ManualBagging với Random Forest thay đổi đồng thời cách xét đặc trưng và quy tắc tổng hợp. Vì vậy, chưa thể quy toàn bộ chênh lệch kết quả cho một thành phần. Báo cáo bổ sung thiết kế 2 × 2 để tách hai yếu tố trong điều kiện chung.

Bảng 4.5. Thiết kế đối chứng 2 × 2

Xét đặc trưng tại nút	Tổng hợp	Mục đích đối chiếu
Toàn bộ	Hard voting	Mốc tham chiếu
Toàn bộ	Soft voting	Thay riêng tổng hợp
sqrt	Hard voting	Thay riêng xét đặc trưng
sqrt	Soft voting	Kết hợp hai lựa chọn

Thí nghiệm chỉ dùng 426 quan sát train của seed 42; 143 quan sát test được giữ bên ngoài. Cả bốn cấu hình dùng 51 cây và cùng năm fold ngoài. Trong phần học của mỗi fold ngoài, CV năm fold chọn độ sâu của cây đơn trong các ứng viên 2–6; sau đó dùng chung độ sâu cho bốn cấu hình. Các độ sâu lần lượt là 2, 6, 2, 2 và 6.

Khi đổi cách xét đặc trưng, chương trình giữ nguyên các chỉ số bootstrap và seed của từng cây. Trong cùng nhóm đặc trưng, hard và soft sử dụng chính những cây đã học, chỉ thay bước tổng hợp. Do đó, các chênh lệch được đối chiếu trên cùng fold và tại độ sâu được kiểm soát.

Thuật toán 4.2. Đối chứng đặc trưng và quy tắc tổng hợp

Đầu vào: 426 quan sát train; B = 51; 5 fold ngoài.
1. Chia train thành 5 fold ngoài có phân tầng.
2. Trong từng fold, chọn độ sâu bằng CV trên phần học.
3. Sinh và lưu B bootstrap cùng seed của từng cây.
4. Trên mỗi bootstrap, học hai cây: toàn bộ đặc trưng và sqrt; giữ nguyên seed và độ sâu tương ứng.
5. Với mỗi nhóm cây, dự đoán fold ngoài bằng hard/soft.
6. Ghi chỉ số tổ hợp, chất lượng từng cây và bất đồng.
7. Tổng hợp trung bình, SD và chênh lệch cùng fold.
Đầu ra: Bảng đối chứng 4 cấu hình và phân tích đa dạng.

Đây là phân tích bổ sung sau thí nghiệm gốc, chưa tối ưu riêng từng biến thể. Nhóm sqrt được gọi là Bagging có ngẫu nhiên hóa đặc trưng, chưa đồng nhất với RandomForestClassifier. SD giữa các fold là thống kê mô tả, không phải khoảng tin cậy.

4.10 Thiết kế so sánh các hướng Ensemble Learning

Phần mở rộng sử dụng đúng mười lần chia dữ liệu phân tầng theo tỷ lệ 75/25, với các seed từ 42 đến 51 như thí nghiệm gốc. Trong mỗi seed, mọi mô hình sử dụng chung 426 quan sát train và 143 quan sát test. Ba kết quả gốc được giữ để đối chiếu; bảy cấu hình mới được khai báo trước lần chạy bổ sung và không được điều chỉnh theo điểm test.

Bộ mô hình cơ sở của Voting và Stacking gồm cây quyết định độ sâu 3 cố định, Logistic Regression có StandardScaler trong Pipeline và Gaussian Naive Bayes. Ba mô hình này được đánh giá riêng nhằm kiểm tra lợi ích của việc kết hợp.

Cây độ sâu 3 được ghi thành một dòng riêng vì khác với quy trình chọn độ sâu bằng đánh giá chéo của cây đơn trong thí nghiệm gốc. Việc cố định kiến trúc trước khi tạo OOF giúp không sử dụng fold được giữ để lựa chọn cấu hình cây.

Bảng 4.6. Cấu hình phần mở rộng các họ ensemble

Nhóm	**Cấu hình cố định**
AdaBoost	Tối đa 51 cây độ sâu 1; learning_rate = 1; SAMME
Voting hard/soft	Cây độ sâu 3 + Logistic Regression chuẩn hóa + Gaussian NB; trọng số bằng nhau

Nhóm	Cấu hình cố định
Stacking	Cùng ba mô hình; OOF 5 fold phân tầng có trộn; predict_proba; tầng hai Logistic Regression; passthrough=False
Đối chứng cơ sở	Đánh giá riêng cây độ sâu 3, Logistic Regression và Gaussian NB

Đây là khảo sát bổ sung sau khi đã quan sát kết quả thí nghiệm gốc, nên được diễn giải như một đối chiếu mô tả, không phải xác nhận độc lập trên dữ liệu mới.

Các mô hình chưa được tối ưu riêng với cùng ngân sách tính toán. Bảng kết quả so sánh các cấu hình đã công bố, không tách riêng tác động nhân quả của từng họ phương pháp. Trung bình và độ lệch chuẩn mẫu sử dụng cùng quy ước với mục 4.7.

CHƯƠNG 5. TRIỂN KHAI, KẾT QUẢ VÀ THẢO LUẬN

5.1 Môi trường và tổ chức mã nguồn

Bảng 5.1. Môi trường thực nghiệm

Thành phần	Phiên bản
Python	3.12.14
NumPy	2.3.5
scikit-learn	1.8.0
Matplotlib	3.10.8

NumPy hỗ trợ xử lý mảng và lấy mẫu. Scikit-learn cung cấp dữ liệu, cây cơ sở, Random Forest và công cụ đánh giá. Matplotlib tạo biểu đồ.

Bảng 5.2. Tổ chức mã nguồn

Thành phần	Chức năng
ensemble_model.py	Lớp ManualBagging
de_5_ensemble_learning.py	Quy trình thí nghiệm gốc
examples/toy_bagging.py	Ví dụ tính tay
experiments/report_extensions.py	Khảo sát số cây và phân tích lỗi
tests/	Kiểm thử
outputs/	Kết quả và hình
experiments/controlled_comparison.py	Đối chứng đặc trưng, hard/soft và so sánh theo cặp
experiments/ensemble_families.py	So sánh AdaBoost, Voting và Stacking với các mô hình gốc và mô hình cơ sở

Các đoạn code dưới đây được trích lược để giải thích. Mã đầy đủ đi kèm trong repository và bao gồm kiểm tra dữ liệu, tham số, trạng thái mô hình cùng chức năng xuất tệp.

5.2 Triển khai bootstrap và huấn luyện

Phần cốt lõi trong ManualBagging.fit:

```
self.classes_, encoded = np.unique(y, return_inverse=True)
rng = np.random.default_rng(self.random_state)

self.estimators_ = []
self.estimators_samples_ = []

n_samples = len(X)
oob_votes = np.zeros(
    (n_samples, len(self.classes_)),
    dtype=np.int64,
)

for _ in range(self.n_estimators):
    indices = rng.integers(
        0, n_samples, size=n_samples
    )

    tree = DecisionTreeClassifier(
        max_depth=self.max_depth,
        random_state=int(rng.integers(0, 2**31 - 1)),
    )

    tree.fit(X[indices], encoded[indices])

    self.estimators_.append(tree)
    self.estimators_samples_.append(indices)

    if self.oob_score:
        unseen = np.flatnonzero(
            np.bincount(indices, minlength=n_samples) == 0
        )

        if len(unseen):
            predicted = tree.predict(X[unseen]).astype(int)
            np.add.at(oob_votes, (unseen, predicted), 1)
```

np.unique tạo thứ tự các lớp và ánh xạ nhãn sang chỉ số liên tiếp. Điều này giúp đếm phiếu đúng khi nhãn gốc là chuỗi hoặc số không liên tiếp.

rng.integers sinh N chỉ số có thể lặp. X và nhãn cùng sử dụng indices, bảo đảm đặc trưng không bị lệch nhãn.

Cây được lưu trong estimators_. Chỉ số bootstrap được lưu để kiểm tra và xác định những mẫu OOB.

Cây cơ sở dùng thư viện, nhưng việc lấy mẫu, tổ chức nhiều cây và quản lý phiếu được triển khai trực tiếp.

5.3 Triển khai hard voting

Phần đếm phiếu:

```
counts = np.zeros(
    (len(X), len(self.classes_)), dtype=np.int64
)
rows = np.arange(len(X))
for tree in self.estimators_:
    predicted = tree.predict(X).astype(int)
    np.add.at(counts, (rows, predicted), 1)
```

Mỗi hàng ứng với một quan sát; mỗi cột ứng với một lớp. Mỗi cây tăng đúng một phiếu cho mỗi quan sát.

Sau đó:

```
predicted_labels = self.classes_[counts.argmax(axis=1)]
```

5.4 Triển khai OOB

Sau khi tích luỹ phiếu, chương trình xử lý:

```
self.oob_vote_counts_ = oob_votes
self.oob_counts_ = oob_votes.sum(axis=1)

covered = self.oob_counts_ > 0
self.oob_coverage_ = float(covered.mean())

self.oob_decision_function_ = np.full(
    oob_votes.shape,
    np.nan,
)

self.oob_decision_function_[covered] = (
    oob_votes[covered]
    / self.oob_counts_[covered, None]
)

self.oob_score_ = (
    float(
        accuracy_score(
            encoded[covered],
            oob_votes[covered].argmax(axis=1),
        )
    )
    if covered.any()
    else float("nan")
)
```

covered xác định các mẫu có ít nhất một phiếu. Chỉ những mẫu này tham gia accuracy OOB.

Các hàng không có phiếu giữ giá trị NaN trong ma trận tỷ lệ phiếu OOB. Cách làm tránh việc argmax trên một hàng toàn 0 tạo ra dự đoán lớp đầu một cách không hợp lệ.

Độ sâu của mô hình gốc đã được chọn trên train trước khi tính OOB. Vì vậy, OOB sau lựa chọn này không hoàn toàn độc lập với tuning và được sử dụng như đánh giá bổ sung.

5.5 Triển khai lựa chọn độ sâu và khảo sát số cây

5.5.1 Chọn độ sâu

Phần lựa chọn trong chương trình:

```
def select_depth(X_train, y_train, seed):
    folds = StratifiedKFold(
        n_splits=5, shuffle=True, random_state=seed
    )
    scores = {}
    for depth in (2, 3, 4, 5, 6):
        values = cross_val_score(
            DecisionTreeClassifier(
                max_depth=depth, random_state=seed
            ),
            X_train, y_train, cv=folds,
            scoring="balanced_accuracy", n_jobs=1,
        )
        scores[depth] = float(values.mean())
    return max(scores, key=scores.get), scores
```

Hàm chỉ nhận train. Các ứng viên được đưa vào theo thứ tự tăng dần nên khi hòa điểm, độ sâu nhỏ hơn được giữ trước.

5.5.2 Khảo sát số cây

Phần cốt lõi của khảo sát bổ sung:

```
folds = StratifiedKFold(
    n_splits=5, shuffle=True, random_state=42
)
for fold, (fit, validation) in enumerate(
    folds.split(X_train, y_train), start=1
):
    depth, _ = select_depth(
        X_train[fit], y_train[fit], seed=42 + fold
    )
    for count in (1, 5, 11, 21, 51, 101):
        model = ManualBagging(
            n_estimators=count, max_depth=depth,
            random_state=42 + fold, oob_score=False,
        )
        model.fit(X_train[fit], y_train[fit])
        predicted = model.predict(X_train[validation])
        score = balanced_accuracy_score(
            y_train[validation], predicted
        )
```

Việc chọn độ sâu nằm bên trong phần học của từng fold ngoài. Fold ngoài được giữ lại chỉ dùng để đánh giá.

Trong cùng fold, seed và độ sâu giống nhau giữa các B. Các mô hình vì vậy sử dụng cùng chuỗi bootstrap tương ứng, khác số lượng cây được lấy từ chuỗi đó.

5.5.3 Chạy thí nghiệm gốc

Quy trình gốc được gọi qua hàm run_experiment:

```
from sklearn.datasets import load_breast_cancer
from de_5_ensemble_learning import run_experiment

data = load_breast_cancer()
for seed in range(42, 52):
    result = run_experiment(data, seed=seed, n_estimators=51)
    print(
        "Seed:", seed,
        "Selected depth:", result.depth,
        "Metrics:", result.metrics,
    )
```

Hàm này thực hiện chia dữ liệu, chọn độ sâu, học ba mô hình và đánh giá. Chương trình đầy đủ còn tổng hợp các seed và xuất bảng, hình.

5.6 Kiểm thử chương trình

Bộ kiểm thử tự động kiểm tra các bước cốt lõi của Bagging, tổng hợp hard/soft, ánh xạ xác suất khi cây chỉ học một lớp, phép đo bất đồng và giao thức so sánh các họ ensemble. Các kiểm thử của phần mở rộng còn xác minh việc chuẩn hóa trong

từng fold, tính nhất quán của các mô hình cơ sở giữa hard và soft voting, và việc loại tập test khỏi các fold OOF. Chạy toàn bộ bộ kiểm thử trước khi tái lập kết quả.

Bảng 5.3. Các nhóm kiểm thử

Nhóm	**Nội dung**
Dữ liệu và tham số	Kích thước, giá trị không hợp lệ, tham số sai
Trạng thái mô hình	Dự đoán trước huấn luyện
Bootstrap	Đúng số lượt, có lặp, tái lập
Voting	Tổng phiếu, ánh xạ nhãn, hòa phiếu
OOB	Đúng nguồn phiếu, coverage, mẫu không có phiếu
Quy trình gốc	Chọn độ sâu chỉ dùng train
Kết quả và xuất tệp	Hồi quy seed 42, CLI thực, JSON, CSV, PNG
Ví dụ tính tay	Ngưỡng, Gini, phiếu và OOB
Khảo sát bổ sung	Chọn độ sâu không nhận fold ngoài đánh giá
Truy vết cây	Đúng nhánh, lá và dự đoán
Đối chứng hard/soft	Khác biệt nhãn, cây học một lớp, ánh xạ xác suất và bất đồng

Kiểm thử giúp phát hiện lỗi triển khai và hỗ trợ tái lập. Việc đạt kiểm thử không thay thế đánh giá khả năng dự đoán trên dữ liệu ngoài.

5.7 Kết quả lựa chọn độ sâu

Bảng 5.4. Balanced accuracy CV tại seed 42

Độ sâu	**Balanced accuracy**
2	91,75%
3	93,73%
4	92,42%
5	92,55%
6	92,05%

Độ sâu 3 có điểm trung bình cao nhất nên được chọn. Trong các ứng viên, tăng độ sâu không làm điểm CV tăng liên tục.

5.8 Kết quả tại seed 42

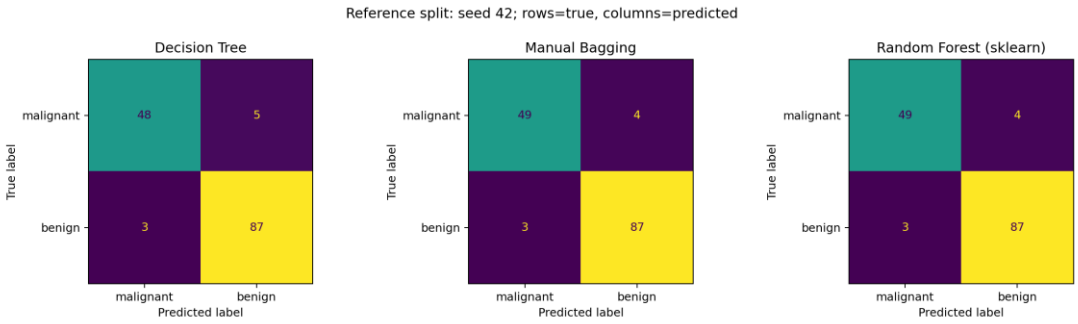
Bảng 5.5. Kết quả trên tập test

Mô hình	Accuracy	Precision ác	Recall ác	F1 ác
Cây đơn	94,41%	94,12%	90,57%	92,31%
Manual Bagging	95,10%	94,23%	92,45%	93,33%
Random Forest	95,10%	94,23%	92,45%	93,33%

Cây đơn dự đoán đúng 135/143 mẫu, Bagging đúng 136/143. Chênh lệch một mẫu tương đương khoảng 0,70 điểm phần trăm accuracy.

Bảng 5.6. Số đếm phân loại

Mô hình	TP	FN	FP	TN
Cây đơn	48	5	3	87
Manual Bagging	49	4	3	87
Random Forest	49	4	3	87



Hình 5.1. Ma trận nhầm lẫn của ba mô hình tại seed 42.

Hàng là nhãn thật, cột là dự đoán; thứ tự hai trục là ác tính rồi lành tính.

Bảng 5.7. Kiểm tra cách tính chỉ số Bagging

Chỉ số	Thay số	Kết quả
Accuracy	$\frac{49 + 87}{143}$	95,10%
Precision ác	$\frac{49}{49 + 3}$	94,23%
Recall ác	$\frac{49}{49 + 4}$	92,45%
F1 ác	$\frac{98}{98 + 3 + 4}$	93,33%
Specificity	$\frac{87}{87 + 3}$	96,67%
Balanced accuracy	$\frac{\frac{49}{53} + \frac{87}{90}}{2}$	94,56%

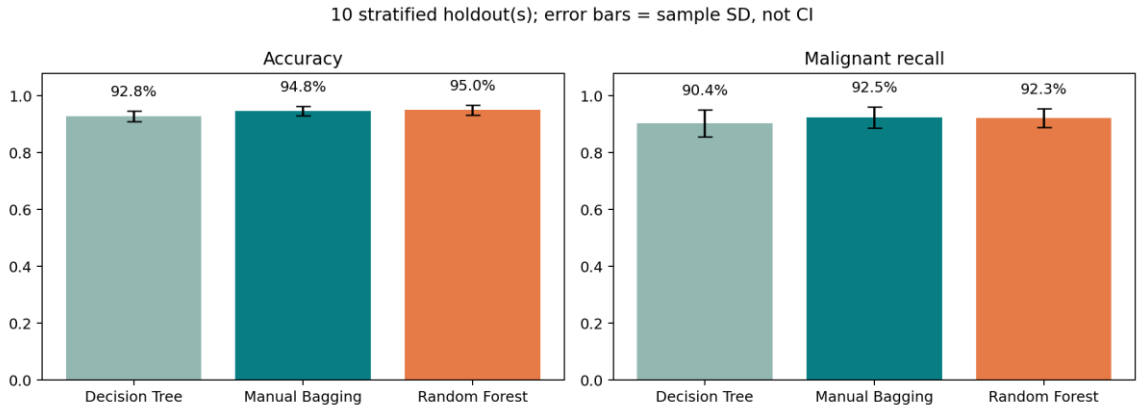
Bagging có số ca bỏ sót ác tính giảm từ năm xuống bốn so với cây đơn, trong khi FP đều bằng ba. Đây là nhận xét về số lượng lỗi của lần chạy, chưa chứng minh ưu thế tổng quát.

5.9 Kết quả trên mười lần chia

Bảng 5.8. Kết quả tổng hợp, trung bình ± SD mẫu

Mô hình	Accuracy (%)	Recall ác (%)	F1 ác (%)
Cây đơn	92,80 ± 1,84	90,38 ± 4,66	90,30 ± 2,42
Manual Bagging	94,76 ± 1,59	92,45 ± 3,77	92,89 ± 2,13
Random Forest	95,03 ± 1,85	92,26 ± 3,26	93,26 ± 2,39

Trung bình tính theo %, SD tính theo điểm phần trăm.



Hình 5.2. Accuracy và recall trung bình trên mười lần chia dữ liệu.

Bagging có accuracy trung bình cao hơn cây đơn khoảng 1,96 điểm phần trăm; recall và F1 trung bình cũng cao hơn. SD accuracy của Bagging thấp hơn trong những lần chia đã khảo sát.

Trong thí nghiệm gốc, Random Forest có accuracy trung bình cao hơn Bagging khoảng 0,28 điểm phần trăm. Ngược lại, recall ác tính của Bagging cao hơn khoảng 0,19 điểm phần trăm. Vì vậy, không có một mô hình thắng ở mọi chỉ số.

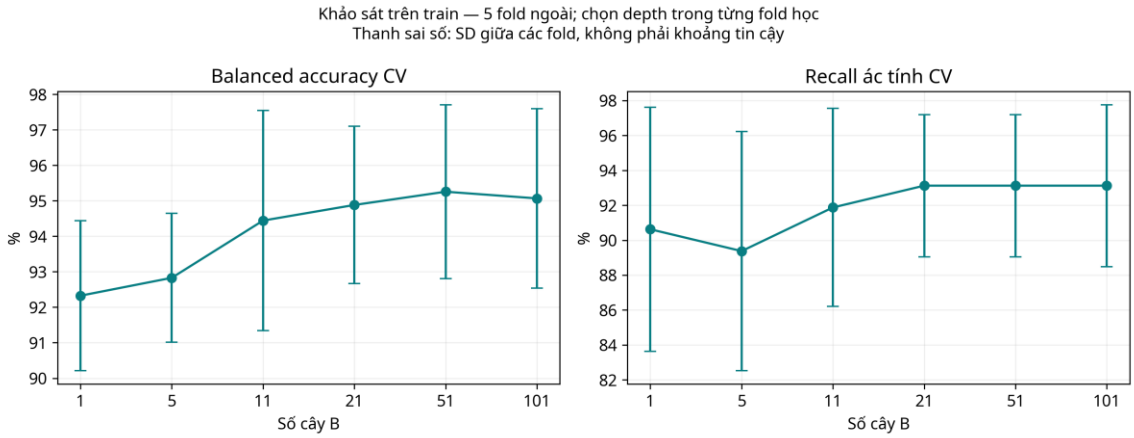
Các chênh lệch cần được đọc cùng mức dao động và giới hạn của các holdout giao nhau. Báo cáo chưa thực hiện kiểm định để khẳng định khác biệt có ý nghĩa thống kê.

5.10 Kết quả khảo sát số cây

Bảng 5.9. Khảo sát số cây bằng CV trên train

Số cây	Accuracy CV trung bình (%)	Balanced accuracy CV trung bình ± SD
1	92,72	92,33 ± 2,11
5	93,66	92,83 ± 1,82
11	95,08	94,44 ± 3,11
21	95,31	94,88 ± 2,22
51	95,78	95,26 ± 2,45
101	95,54	95,07 ± 2,53

Trung bình tính theo %, SD tính theo điểm phần trăm.



Hình 5.3. Ảnh hưởng số cây trong CV trên train; thanh sai số là SD giữa các fold.

Balanced accuracy trung bình tăng từ 92,33% với một cây lên 95,26% với 51 cây trong khảo sát. Tăng tiếp lên 101 cây không cải thiện, điểm giảm nhẹ còn 95,07%.

Các chỉ số không tăng đồng loạt theo B. Chẳng hạn, recall trung bình ở năm cây thấp hơn ở một cây. Điều này cho thấy tác động của số cây cần được kiểm tra thay vì giả định tăng đơn điệu.

Trong sáu cấu hình, 51 cây có balanced accuracy CV trung bình cao nhất. Kết quả bổ sung hỗ trợ cấu hình ban đầu, chưa chứng minh 51 tối ưu tổng quát.

Cần phân biệt:

- 95,26% là balanced accuracy CV của khảo sát trên train.
- 95,10% là accuracy test tại seed 42.
- 94,76% là accuracy trung bình của mười holdout.

Ba số thuộc ba quy trình đánh giá khác nhau, không được sử dụng thay thế.

5.11 Kết quả OOB

Bảng 5.10. OOB của Bagging tại seed 42

Nội dung	Kết quả
Số mẫu train	426
Số mẫu có phiếu OOB	426
Coverage	100%
Số dự đoán đúng	401
Accuracy OOB	94,13%
Số cây OOB mỗi mẫu	8–34

Accuracy OOB bằng 401/426. Coverage 100% chỉ có nghĩa mọi mẫu nhận ít nhất một phiếu hợp lệ.

Số cây OOB khác nhau giữa các mẫu do mỗi mẫu xuất hiện hoặc vắng mặt trong những bootstrap khác nhau.

OOB và test đánh giá trên các quan sát và tập cây bỏ phiếu khác nhau. Đồng thời, OOB được tính sau chọn độ sâu nên không hoàn toàn độc lập với tuning. Vì vậy, điểm OOB được sử dụng bổ sung cho kết quả test.

5.12 Phân tích trường hợp dự đoán sai

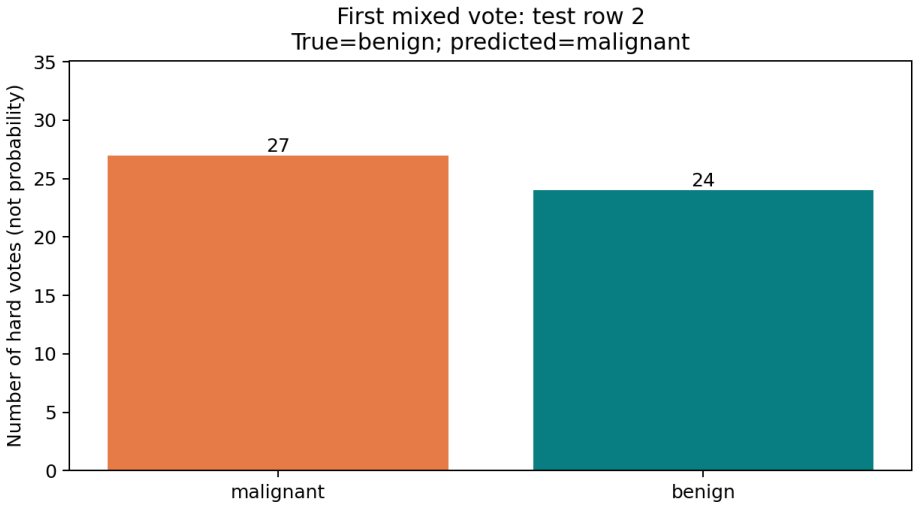
5.12.1 Thông tin quan sát

Chương trình chọn quan sát test đầu tiên có phiếu ở cả hai lớp. Quan sát có chỉ số test bằng 2 (chỉ số bắt đầu từ 0), tức quan sát thứ ba, tương ứng hàng 291 trong bộ dữ liệu.

Nhãn thật là lành tính. Bagging cho:

- 27 phiếu ác.
- 24 phiếu lành.

Hard voting chọn ác, tạo dương tính giả.



Hình 5.4. Trường hợp bất đồng với 27 phiếu ác và 24 phiếu lành.

Biên phiếu:

$$\frac{|27 - 24|}{51} = 0{,}0588 \approx 5{,}88\%.$$

Chênh lệch giữa tỷ lệ phiếu ác và lành là 5,88 điểm phần trăm. Các cây khá bất đồng; tỷ lệ phiếu và biên phiếu chưa phải xác suất bệnh hoặc độ chắc chắn đã được hiệu chỉnh.

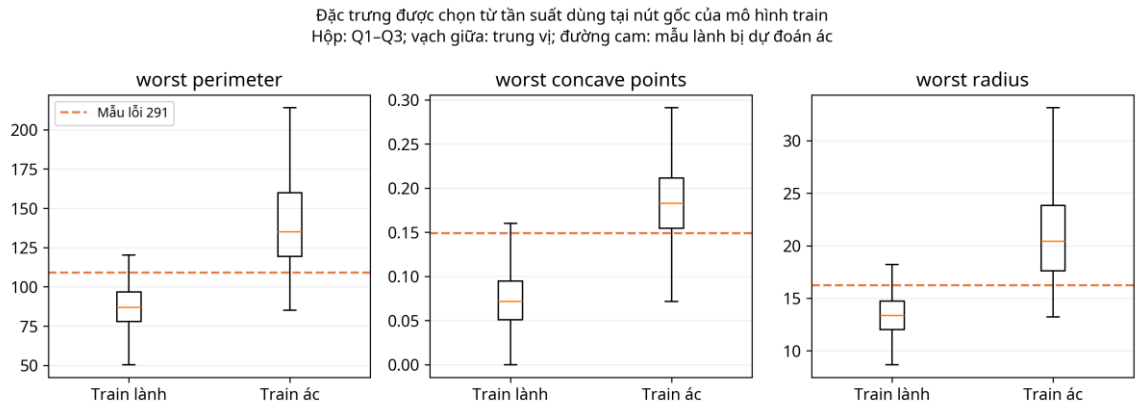
5.12.2 Đặc điểm mẫu so với train

Ba đặc trưng minh họa được chọn từ tần suất sử dụng tại nút gốc của mô hình đã học trên train. Các đặc trưng worst perimeter, worst concave points, worst radius được dùng tại nút gốc lần lượt ở 16, 14 và 9 cây.

Bảng 5.11. Đặc điểm mẫu 291 so với hai lớp trên train

Đặc trưng	**Mẫu 291**	**Trung vị train lành**	**Trung vị train ác**
worst perimeter	109,10	86,82	135,10
worst concave points	0,14890	0,07174	0,18270
worst radius	16,25	13,35	20,42

Ở cả ba đặc trưng, mẫu có giá trị lớn hơn trung vị nhóm lành và nhỏ hơn trung vị nhóm ác. Các giá trị còn nằm trên Q3 của nhóm lành và dưới Q1 của nhóm ác.



Hình 5.5. Mẫu 291 so với phân bố train; hộp Q1–Q3, vạch giữa là trung vị, nét đứt là mẫu lỗi.

Phân tích này cho thấy vị trí của mẫu giữa các vùng trung tâm của phân bố trên những đặc trưng được xem xét. Tuy nhiên, ba đặc trưng chưa đủ xác định ranh giới quyết định trong không gian 30 chiều và không giải thích nguyên nhân y khoa.

5.12.3 Truy vết hai cây có dự đoán khác nhau

Cây thứ nhất đi qua các điều kiện:

1. worst radius = 16,25 ≤ 16,79500.
2. worst concave points = 0,14890 > 0,14315.
3. worst symmetry = 0,29620 > 0,28855.

Mẫu đến một lá có 18 lượt hàng bootstrap, tất cả nhãn ác. Vì vậy, cây thứ nhất dự đoán ác. Đây không nhất thiết là 18 quan sát khác nhau do bootstrap có thể lặp hàng.

Cây thứ hai đi qua:

1. mean concave points = 0,04819 ≤ 0,04989.
2. worst symmetry = 0,29620 > 0,16385.
3. worst perimeter = 109,10 ≤ 127,20.

Lá tương ứng chứa 259 lượt hàng bootstrap: 256 lượt nhãn lành và 3 lượt nhãn ác. Tỷ lệ nhãn lành khoảng 98,84%, nên cây thứ hai dự đoán lành.

Các ngưỡng được làm tròn để trình bày. Dữ liệu truy vết lưu giá trị đầy đủ.

Hai cây đã học những đặc trưng và ngưỡng khác nhau trên các bootstrap khác nhau. Việc truy vết giải thích trực tiếp vì sao cùng một mẫu nhận hai phía phiếu.

Độ thuần của lá trên bootstrap không bảo đảm dự đoán đúng quan sát mới. Truy vết hai cây chỉ phục vụ minh họa; kết quả cuối vẫn sử dụng đủ 51 phiếu.

5.12.4 Cùng điểm tổng hợp không có nghĩa cùng lỗi

Bagging và Random Forest dự đoán khác nhau ở đúng hai quan sát test:

Bảng 5.12. Các quan sát có dự đoán khác nhau

Hàng dataset	Nhãn thật	Bagging	Random Forest
291	Lành	Ác — sai	Lành — đúng
128	Lành	Lành — đúng	Ác — sai

Hai mô hình vẫn có cùng TP, FN, FP, TN và accuracy ở seed 42 vì các lỗi dương tính giả xuất hiện trên những quan sát khác nhau.

Do đó, không thể kết luận hai mô hình tương đương chỉ từ chỉ số tổng hợp. Việc Random Forest dự đoán đúng riêng mẫu 291 cũng chưa chứng minh phương pháp này luôn tốt hơn Bagging.

5.13 Đối chứng cách xét đặc trưng và quy tắc tổng hợp

5.13.1 Kết quả bốn cấu hình

Bảng 5.13. Kết quả đối chứng trên train

Xét đặc trưng	Tổng hợp	Accuracy CV (%)	Balanced accuracy CV (%)
Toàn bộ	Hard	95,78	95,26 ± 2,45
Toàn bộ	Soft	95,78	95,26 ± 2,45
sqrt	Hard	94,37	93,62 ± 3,48
sqrt	Soft	94,13	93,44 ± 3,77

Số sau dấu ± là SD mẫu giữa năm fold, tính theo điểm phần trăm. Các hàng là trung bình chỉ số của từng fold, không phải chỉ số gộp từ toàn bộ dự đoán. Nguồn: outputs/extensions/controlled_comparison_summary.csv.

Với các cây xét toàn bộ đặc trưng, hard và soft cho cùng nhãn ở cả 426 lượt dự đoán ngoài fold. Trong nhóm sqrt, hai quy tắc khác nhau ở một quan sát; balanced

accuracy trung bình của soft thấp hơn khoảng 0,19 điểm phần trăm. Việc thay quy tắc tổng hợp chưa mang lại cải thiện trong cấu hình khảo sát, chưa chứng minh hai quy tắc luôn tương đương hoặc soft luôn kém hơn [3].

Khi giữ hard voting và đổi từ toàn bộ đặc trưng sang sqrt, balanced accuracy trung bình giảm khoảng 1,63 điểm phần trăm. Chênh lệch được tính trong từng fold rồi lấy trung bình. Báo cáo chưa thực hiện kiểm định để khẳng định ý nghĩa thống kê của mức chênh lệch này.

5.13.2 Chất lượng cây cơ sở và mức bất đồng

Gọi v_ic là số cây dự đoán lớp c cho quan sát i, B là số cây và M là số quan sát của fold đánh giá. Mức bất đồng trung bình giữa các cặp cây được tính:

$$D = \frac{1}{M}\sum_{i=1}^{M}\frac{B^2 - \sum_{c=1}^{C} v_{ic}^2}{B(B-1)}. \quad (5.1)$$

D được tính trong mỗi fold rồi lấy trung bình giữa năm fold. Với hai lớp và 51 cây, D lớn nhất khoảng 50,98% khi phiếu chia 26:25. D đo tỷ lệ các cặp cây dự đoán khác nhãn; không trực tiếp đo tương quan lỗi, độc lập thống kê hay xác suất mắc bệnh.

Bảng 5.14. Chất lượng cây cơ sở và tính đa dạng

Xét đặc trưng	Balanced accuracy trung bình một cây (%)	Bất đồng D (%)
Toàn bộ	91,67	7,99
sqrt	89,75	10,52

Balanced accuracy của từng cây được tính trên fold ngoài, sau đó lấy trung bình qua các cây và các fold. Ngẫu nhiên hóa đặc trưng làm D tăng từ 7,99% lên 10,52%, đồng thời chất lượng cây cơ sở giảm từ 91,67% xuống 89,75%. Balanced accuracy tổ hợp hard cũng giảm từ 95,26% xuống 93,62%.

Đối chứng cho thấy đa dạng hơn chưa đủ để tổ hợp tốt hơn: lợi ích từ khác biệt giữa các cây cần được xem cùng chất lượng từng cây. Đây là quan sát trong thiết kế hiện tại, chưa thể suy rộng rằng xét toàn bộ đặc trưng luôn tốt hơn ngẫu nhiên hóa đặc trưng. Việc chọn độ sâu chung bằng cây đơn cũng là giới hạn khi diễn giải.

5.13.3 Mã cốt lõi của phép đối chứng

Đoạn mã trích lược từ experiments/controlled_comparison.py tổng hợp từ cùng những cây đã học nhãn mã hóa chung. Nếu một bootstrap chỉ chứa một lớp, cột xác suất vẫn được ánh xạ qua tree.classes_ để tránh cộng sai lớp:

```
def aggregate_trees(trees, X, n_classes=2):
    votes = np.zeros((len(X), n_classes), dtype=np.int64)
    probabilities = np.zeros((len(X), n_classes))
    rows = np.arange(len(X))
    for tree in trees:
        predicted = tree.predict(X).astype(int)
        np.add.at(votes, (rows, predicted), 1)
        columns = tree.classes_.astype(int)
        probabilities[:, columns] += tree.predict_proba(X)
    return {
        "votes": votes,
        "mean_probabilities": probabilities / len(trees),
        "hard": votes.argmax(axis=1),
        "soft": probabilities.argmax(axis=1),
    }
```

Do mọi cây có cùng trọng số, chọn lớp từ tổng xác suất và từ trung bình xác suất cho cùng kết quả argmax. Mã đầy đủ còn giữ nguyên bootstrap và seed khi thay max_features, lưu thông tin fold và tính chênh lệch theo cặp.

5.14 So sánh theo cặp trên mười lần chia gốc

Với mỗi seed s, tính chênh lệch accuracy của Bagging với mô hình đối chiếu trong cùng lần chia:

$$\Delta_s = \text{Accuracy}_{\text{Bagging},s} - \text{Accuracy}_{\text{đối chiếu},s}. \quad (5.2)$$

Bảng 5.15. So sánh accuracy theo cặp giữa mười seed

Đối chiếu	Cao hơn	Bằng	Thấp hơn	Chênh lệch trung bình
Bagging − cây đơn	9	1	0	+1,96 đpt
Bagging − RF	2	4	4	−0,28 đpt

Đpt là điểm phần trăm. Bagging có accuracy cao hơn cây đơn ở chín trong mười lần chia và bằng nhau ở lần còn lại. Kết quả cung cấp bằng chứng mô tả về mức nhất quán của cải thiện, bổ sung cho bảng trung bình. So với Random Forest, Bagging chưa thể hiện ưu thế nhất quán.

Các tập test giữa các seed có thể giao nhau nên không coi đây là mười phép thử độc lập và không suy ra giá trị p từ số lần thắng. Phân tích giữ nguyên kết quả gốc, không điều chỉnh cấu hình từ các bảng đối chiếu này.

5.15 Thảo luận chung

Về thuật toán, việc triển khai tường minh bootstrap, voting và OOB giúp kiểm tra được nguồn dữ liệu của từng cây và nguồn phiếu của từng quan sát.

Về kết quả, Bagging đạt mức trung bình cao hơn cây đơn trong thí nghiệm gốc. Khảo sát bổ sung cho thấy tăng số cây có thể đem lại lợi ích, nhưng kết quả không tăng ở mọi chỉ số và không tiếp tục cải thiện khi tăng từ 51 lên 101 cây trong thiết kế đã chạy.

Về khả năng giải thích, truy vết cây cung cấp bằng chứng cụ thể hơn so với chỉ báo cáo số phiếu. Tuy nhiên, giải thích một số nhánh chưa tương đương với xác định nguyên nhân tổng quát của mọi lỗi.

Nếu T_tree là chi phí học một cây, huấn luyện tuần tự B cây có chi phí xấp xỉ B × T_tree. Với K quan sát và độ sâu tối đa d, duyệt cây khi dự đoán có bậc O(KBd). Ma trận phiếu có kích thước K × C; mã còn lưu B × N chỉ số bootstrap.

Đây là phân tích cấu trúc, chưa phải đo thời gian. Cần cân nhắc mức cải thiện với chi phí học và lưu mô hình. Việc 101 cây chưa cải thiện so với 51 trong khảo sát không chứng minh 51 tối ưu cho mọi dữ liệu.

5.16 Triển khai và kết quả của AdaBoost, Voting, Stacking

5.16.1 Tổ chức triển khai

Tệp experiments/ensemble_families.py dựng các cấu hình bằng scikit-learn, lưu chỉ số của từng seed và tổng hợp kết quả. Đoạn mã sau minh họa cách gọi các mô hình từ hàm build_models; đầy đủ cấu hình và cách tạo OOF nằm trong mã nguồn đi kèm.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from experiments.ensemble_families import build_models

data = load_breast_cancer()

X_train, X_test, y_train, y_test = train_test_split(
    data.data,
    data.target,
    test_size=0.25,
    stratify=data.target,
    random_state=42,
)

models = build_models(seed=42)

for name in (
    "AdaBoost",
    "Voting (hard)",
    "Voting (soft)",
    "Stacking",
):
    model = models[name]
    model.fit(X_train, y_train)

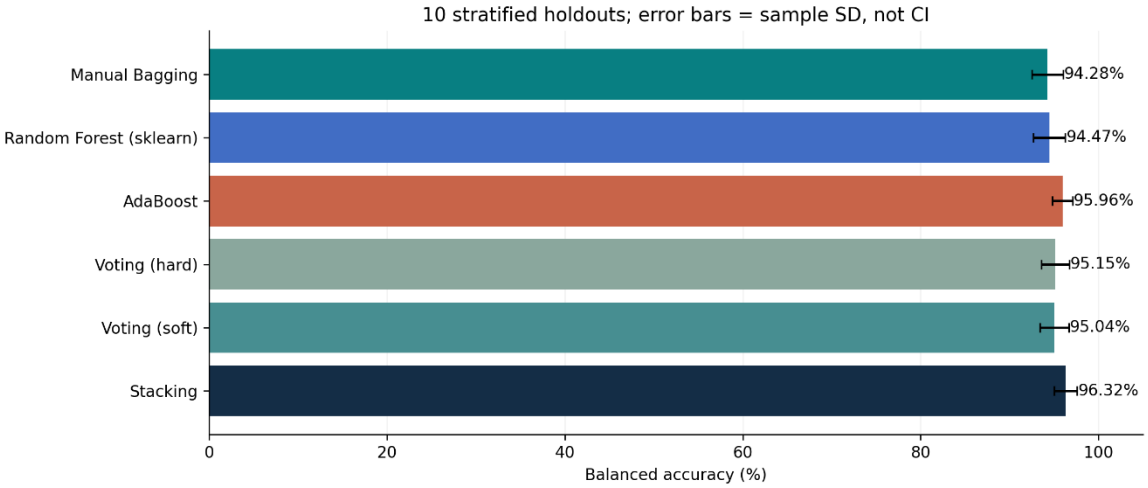
    prediction = model.predict(X_test)
    accuracy = (prediction == y_test).mean()

    print(name, accuracy)
```

Đoạn mã trên minh họa một lần chia dữ liệu tại seed 42. Chương trình đầy đủ lặp lại quy trình trên mười seed để tạo các bảng tổng hợp.

Các chỉ tiêu precision, recall và F1 ác tính trong chương trình sử dụng pos_label bằng 0; ma trận nhầm lẫn giữ thứ tự nhãn [0, 1]. Scikit-learn thực hiện vòng lặp AdaBoost, Voting và OOF Stacking; nhóm tự tổ chức chia dữ liệu, cấu hình, đánh giá và xuất kết quả phục vụ kiểm tra. Phần Bagging vẫn là phần cơ chế tổ hợp do nhóm tự triển khai.

5.16.2 Kết quả trên mười lần chia



Hình 5.6. Balanced accuracy của sáu cấu hình ensemble trên mười lần chia dữ liệu có phân tầng; thanh sai số biểu diễn độ lệch chuẩn mẫu.

Stacking đạt balanced accuracy trung bình cao nhất trong sáu cấu hình ensemble được khảo sát, đạt 96,32%, tiếp theo là AdaBoost với 95,96%. Các kết quả mang tính mô tả trong thiết kế thực nghiệm này; chưa đủ để khẳng định một phương pháp luôn vượt trội.

Bảng 5.16. Kết quả so sánh các cấu hình trên mười holdout

Mô hình	**Accuracy**	**Balanced accuracy**	**Recall ác**
Cây đơn — độ sâu chọn bằng CV	92,80 ± 1,84	92,30 ± 1,94	90,38 ± 4,66
Manual Bagging	94,76 ± 1,59	94,28 ± 1,75	92,45 ± 3,77
Random Forest	95,03 ± 1,85	94,47 ± 1,80	92,26 ± 3,26
Cây đơn — độ sâu 3 cố định	92,94 ± 2,15	92,37 ± 2,41	90,19 ± 4,86
Logistic Regression	97,62 ± 0,82	97,41 ± 0,99	96,60 ± 2,32
Gaussian NB	94,20 ± 1,40	93,37 ± 1,81	90,19 ± 4,15
AdaBoost	96,43 ± 1,07	95,96 ± 1,13	94,15 ± 2,08
Voting hard	95,80 ± 1,23	95,15 ± 1,58	92,64 ± 3,72
Voting soft	95,66 ± 1,27	95,04 ± 1,63	92,64 ± 3,72
Stacking	96,64 ± 1,22	96,32 ± 1,28	95,09 ± 2,70

Trung bình tính theo %, độ lệch chuẩn mẫu tính theo điểm phần trăm, với ddof bằng 1. Mỗi dòng gồm mười seed sử dụng cùng các tập train/test tương ứng. Nguồn

số liệu là outputs/extensions/family_comparison.json và family_comparison_summary.csv. Các chỉ số F1, precision, specificity và số đếm phân loại của từng seed được lưu trong các tệp kết quả.

Trong toàn bộ cấu hình khảo sát, Logistic Regression đạt accuracy trung bình cao nhất, bằng 97,62%. Kết quả này cho thấy các ensemble được xét chưa vượt qua mô hình đơn mạnh nhất. Đây là thứ hạng mô tả của lần khảo sát, chưa phải bằng chứng phương pháp đó luôn tốt nhất.

Voting hard đạt accuracy trung bình 95,80%, Voting soft đạt 95,66%, Stacking đạt 96,64% và AdaBoost đạt 96,43%. Cần đọc các kết quả cùng recall ác tính và mức dao động. Kết hợp thêm mô hình hoặc học thêm tầng hai không tự động bảo đảm cải thiện so với mô hình cơ sở mạnh.

Bagging học các mô hình trên bootstrap; AdaBoost cập nhật trọng số qua các vòng; Voting áp dụng quy tắc kết hợp cố định; Stacking học tầng kết hợp từ các dự đoán OOF. Các cấu hình sử dụng loại mô hình và chi phí tính toán khác nhau, vì vậy bảng kết quả minh họa hành vi của các hướng tiếp cận thay vì chứng minh một cơ chế tổ hợp vượt trội trong mọi điều kiện.

Các tập test giữa các holdout có thể giao nhau; SD chỉ mô tả dao động. Khảo sát này chưa hiệu chuẩn xác suất, chưa thực hiện kiểm định thống kê và chưa đánh giá trên dữ liệu ngoài độc lập.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được

Nhóm đã trình bày Ensemble Learning và tự triển khai bootstrap, hard voting, OOB cho Bagging cây quyết định. Ví dụ năm quan sát được tính từ Gini và ngưỡng chia đến dự đoán, sau đó kiểm chứng bằng Python. AdaBoost, Voting và Stacking được triển khai bổ sung bằng scikit-learn, có công bố cấu hình và kết quả trên cùng các lần chia dữ liệu.

Ba câu hỏi nghiên cứu được trả lời trong phạm vi khảo sát: Bagging có accuracy cao hơn cây đơn ở chín trong mười lần chia và bằng nhau ở lần còn lại; tăng số cây có lợi nhưng chưa cải thiện liên tục; mức bất đồng cao hơn cần được xem cùng chất lượng cây cơ sở.

Phân tích mẫu 291 đã liên hệ vị trí đặc trưng, các nhánh cây và phiếu 27:24 với một lỗi dương tính giả. Mã nguồn cùng cấu hình, kết quả và bộ kiểm thử tự động giúp quá trình triển khai có thể được kiểm tra và tái lập.

6.2 Hạn chế

Trong thí nghiệm gốc, độ sâu được chọn bằng cây đơn rồi dùng chung, chưa tối ưu riêng từng mô hình. Khảo sát số cây và đối chứng chỉ sử dụng train của seed 42; đây là phân tích bổ sung sau thí nghiệm gốc, chưa xác lập cấu hình tối ưu tổng quát. Các cấu hình mới ở mục 4.10 được cố định để minh họa, chưa tối ưu riêng hoặc so sánh theo cùng ngân sách tính toán; phần mở rộng được thực hiện sau khi đã quan sát kết quả gốc.

Các holdout giữa mười seed có thể giao nhau; SD chỉ mô tả dao động, không phải khoảng tin cậy. Báo cáo chưa thực hiện kiểm định để khẳng định ý nghĩa thống kê của những khác biệt và chưa đánh giá trên bộ dữ liệu ngoài độc lập.

Truy vết một số cây giải thích hành vi dự đoán, chưa xác định nguyên nhân y khoa. Tỷ lệ phiếu chưa phải xác suất bệnh đã hiệu chỉnh; kết quả chưa đủ để đánh giá khả năng ứng dụng lâm sàng.

6.3 Hướng phát triển

Các hướng mở rộng ưu tiên gồm:

- Tối ưu cấu hình riêng từng mô hình bằng CV, tiếp tục giữ test ngoài quá trình điều chỉnh.
- Lặp khảo sát số cây và đối chứng hard/soft trên nhiều lần chia train; đánh giá thêm chất lượng xác suất dự đoán.
- Đo thời gian, bộ nhớ và tương quan lỗi giữa các cây; kiểm tra trên dữ liệu ngoài nếu có nguồn phù hợp.

6.4 Kết luận

Bagging có thể cải thiện so với một cây đơn bằng cách kết hợp nhiều mô hình học trên bootstrap, nhưng không loại bỏ mọi sai sót. Các đối chiếu cho thấy cần xem đồng thời số cây, chất lượng mô hình cơ sở và quy tắc tổng hợp. Trong phần mở rộng, Logistic Regression đơn lẻ đạt các chỉ số trung bình cao nhất; kết quả này được giới hạn trong dữ liệu, cấu hình và giao thức đã khảo sát.

Giá trị của báo cáo là mối liên hệ kiểm chứng được giữa nguyên lý, thuật toán, ví dụ, mã nguồn và kết quả. Bagging được giải thích sâu; AdaBoost, Voting và Stacking mở rộng bức tranh học tổ hợp bằng những cơ chế khác nhau. Các kết luận được giới hạn trong giao thức đã công bố.

TÀI LIỆU THAM KHẢO

[1] Breiman, L. (1996). “Bagging Predictors.” Machine Learning, 24, 123–140. https://doi.org/10.1007/BF00058655

[2] Breiman, L. (2001). “Random Forests.” Machine Learning, 45, 5–32. https://doi.org/10.1023/A:1010933404324

[3] Scikit-learn developers. “Ensemble methods.” Scikit-learn User Guide. https://scikit-learn.org/stable/modules/ensemble.html (Truy cập ngày 27/10/2026).

[4] Scikit-learn developers. “load_breast_cancer — Breast Cancer Wisconsin Diagnostic dataset.” Scikit-learn API Reference. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html (Truy cập ngày 27/10/2026).

[5] Scikit-learn developers. “Decision Trees.” Scikit-learn User Guide. https://scikit-learn.org/stable/modules/tree.html (Truy cập ngày 27/10/2026).

[6] Freund, Y., & Schapire, R. E. (1997). “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting.” Journal of Computer and System Sciences, 55(1), 119–139. https://doi.org/10.1006/jcss.1997.1504

[7] Wolpert, D. H. (1992). “Stacked Generalization.” Neural Networks, 5(2), 241–259. https://doi.org/10.1016/S0893-6080(05)80023-1

[8] Nhóm thực hiện. Mã nguồn và kết quả Đề 5 — Ensemble Learning. GitHub: https://github.com/Huyngaongow/hocmaygk. Phiên bản mã và kết quả dùng cho báo cáo, gồm đối chứng bổ sung, được cung cấp trong gói mã nguồn đi kèm.

PHỤ LỤC A. HƯỚNG DẪN TÁI LẬP

Mã nguồn sử dụng Python 3.12; các phiên bản thư viện của lần chạy được liệt kê ở Bảng 5.1. Từ thư mục gốc repository, tạo môi trường ảo, cài các thư viện và chạy chương trình theo thứ tự.

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
.\.venv\Scripts\python.exe de_5_ensemble_learning.py
.\.venv\Scripts\python.exe examples/toy_bagging.py
.\.venv\Scripts\python.exe experiments/report_extensions.py
.\.venv\Scripts\python.exe experiments/controlled_comparison.py
.\.venv\Scripts\python.exe experiments/ensemble_families.py
```

Trên Linux/macOS, tạo môi trường bằng python3 -m venv .venv và thay đường dẫn Python trong các lệnh trên bằng .venv/bin/python.

Chạy thí nghiệm gốc trước các phân tích bổ sung để tạo outputs/results.json. Cấu hình mặc định của thí nghiệm gốc sử dụng 51 cây và các seed từ 42 đến 51. Không sử dụng tùy chọn chạy nhanh khi cần tái lập các bảng tổng hợp trong báo cáo.

Bảng A.1. Các tệp kết quả phục vụ kiểm tra

Tệp/thư mục	**Nội dung kiểm tra**
outputs/results.json	Giao thức, độ sâu, chỉ số và kết quả mười seed của thí nghiệm gốc
outputs/toy/toy_example.json	Gini, ngưỡng, phiếu và OOB của ví dụ tính tay
outputs/extensions/analysis.json	Khảo sát số cây và truy vết mẫu dự đoán sai
controlled_comparison.json	Fold, seed, giao thức và kết quả đối chứng
controlled_comparison_summary.csv	Bảng bốn cấu hình để đối chiếu Bảng 5.13
family_comparison.json	Cấu hình, các lần chia, fold OOF, chỉ số và dự đoán của phần mở rộng
family_comparison_summary.csv	Bảng trung bình và SD để đối chiếu Bảng 5.16

Các tệp controlled_comparison.* và family_comparison.* trong Bảng A.1 nằm trong outputs/extensions/. Số liệu được lấy từ các tệp chương trình sinh; việc làm tròn có thể tạo sai khác ở chữ số cuối. Các nghiên cứu bổ sung không thay đổi kết quả gốc; khảo sát các họ ensemble là đối chiếu mô tả sau thí nghiệm gốc.